



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ
НАУКА У НОВОМ САДУ



Лука Бурсаћ

**Управљање датотекама,
трансакцијама и
интерпретација упита у
релационим базама података**

Нови Сад, 2026



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски - бечелор рад
Аутор, АУ:	Лука Бурсаћ
Ментор, МН:	Др Бранко Милосављевић, редовни професор
Наслов рада, НР:	Управљање датотекама, трансакцијама и интерпретација упита у релационим базама података
Језик публикације, ЈП:	српски/ћирилица
Језик извода, ЈИ:	српски/енглески
Земља публиковања, ЗП:	Република Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2026
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страница/ цитата/табела/слика/графика/прилога)	11/87/9/4/43/0/2
Научна област, НО:	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	Релационе базе података, трансакције, датотеке, слогови, упити
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	Имплементација једног система за управљање релационим базама података у програмском језику Java
Датум прихватања теме, ДП:	
Датум одбране, ДО:	01.01.2025
Чланови комисије, КО:	Председник: Др Петар Петровић, ванредни професор
	Члан: Др Марко Марковић, доцент
	Члан:
	Члан, ментор: Др Бранко Милосављевић, редовни професор
	Потпис ментора

	UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES 21000 NOVI SAD, Trg Dositeja Obradovića 6	
	KEY WORDS DOCUMENTATION	
Accession number, ANO :		
Identification number, INO :		
Document type, DT :	Monographic publication	
Type of record, TR :	Textual printed material	
Contents code, CC :		
Author, AU :	Upisati ime i prezime na latinici	
Mentor, MN :	Igor Dejanović, Phd., full professor	
Title, TI :	Management of files, records and transactions and query interpretation in relational databases	
Language of text, LT :	Serbian	
Language of abstract, LA :	Serbian/English	
Country of publication, CP :	Republic of Serbia	
Locality of publication, LP :	Vojvodina	
Publication year, PY :	2026	
Publisher, PB :	Author's reprint	
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6	
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	11/87/9/4/43/0/2	
Scientific field, SF :	Electrical and Computer Engineering	
Scientific discipline, SD :	Applied computer science and informatics	
Subject/Key words, S/KW :	Relational databases, transactions, files, records, queries	
UC		
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad	
Note, N :		
Abstract, AB :	Implementation of a system for relational database management written in Java	
Accepted by the Scientific Board on, ASB :		
Defended on, DE :	01.01.2025	
Defended Board, DB :	President:	Petar Petrović, Phd., assoc. professor
	Member:	Marko Marković, Phd., asist. professor
	Member:	
	Member, Mentor:	Igor Dejanović, Phd., full professor
		Menthor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Sadržaj

1	Uvod	1
1.1	O sistemu	1
1.2	Klijentsko serverska arhitektura	2
1.3	Sistem za obradu upita	2
2	Upravljanje datotekama	3
2.1	Upravljanje blokovima	3
2.1.1	Interfejs ka <i>file</i> sistemu operativnog sistema	3
2.1.2	Stranice	3
2.1.3	Upravljanje <i>log</i> datotekom	4
2.2	Upravljanje baferima	5
2.2.1	Algoritmi izbora smene bafera	6
2.3	Slogovi	7
2.3.1	Struktuiranje	7
2.4	Primena strukture na blok	8
2.4.1	Stranica slogova	9
3	Upravljanje transakcijama	11
3.1	Realizacija transakcionih mehanizama u sistemu	11
3.1.1	Transakcije kao glavno mesto pristupa vrednostima	11
3.1.2	Sistem za oporavak	12
3.1.3	Bezbedan višenitni pristup	15
3.2	Transakcije i klijenti	18
3.2.1	Algoritam zapisa mirne kontrolne tačke	18
4	Metapodaci	19
4.1	Kataloške tabele	19
4.2	Statistički podaci	20
4.2.1	Računanje statističkih podataka	20
4.3	Pristup metapodacima	20
5	Relacioni operatori	21
5.1	Virtuelna mašina	21
5.1.1	Konstante	21
5.1.2	Izrazi	22
5.1.3	Članovi	23
5.1.4	Predikati	23
5.2	Struktura relacionih operatora u sistemu	23
5.2.1	Pajplajnovano procesovanje	23

5.2.2	Hijerarhija implementacije relacionih operatora	24
5.2.3	Primeri stabla relacionih operatora	28
6	Parsiranje	31
6.1	Tokenizator	31
6.2	Parser	32
6.2.1	Iskazi	32
6.2.2	Sintaktičke kategorije SQL operacija	32
7	Planiranje	39
7.1	Struktura planova u sistemu	39
7.1.1	Hijerarhija implementacije planova	39
7.2	Planer	46
7.2.1	Evaluacija izraza tokom planiranja	46
7.2.2	Ulazna tačka kreiranja i izvršavanja planova	46
7.2.3	Planiranje <i>read-only</i> naredbi	48
7.2.4	Planiranje modifikacionih naredbi	53
8	Klijentsko serverska arhitektura	57
8.1	Komunikacija sa klijentima	57
8.1.1	Odgovori sistema	57
8.1.2	Protokol serijalizacije	58
8.2	Serverski sloj	59
8.2.1	Rukovođenje klijentima	59
8.2.2	Pisanje kontrolnih tačaka	61
8.2.3	Gašenje sistema	61
8.3	Klijentski sloj	62
8.3.1	Štampanje tabela	62
8.3.2	Masovno pokretanje naredbi	62
9	Testovi	63
9.1	Organizacija testova	63
9.1.1	Testno okruženje	63
9.1.2	Sistem izolacije direktorijuma na disku	64
9.1.3	Sistem izolacije direktorijuma u radnoj memoriji	64
10	Pregled sistema	65
10.1	Izgradnja i pokretanje	65
10.1.1	Rukovođenje zavisnostima	65
10.2	Sistemska konfiguracija	66
10.3	Integracija sa <i>GitHub</i> platformom	67
10.4	Primer funkcionisanja celokupnog sistema	67
11	Zaključak	69
11.1	Zbog čega	69

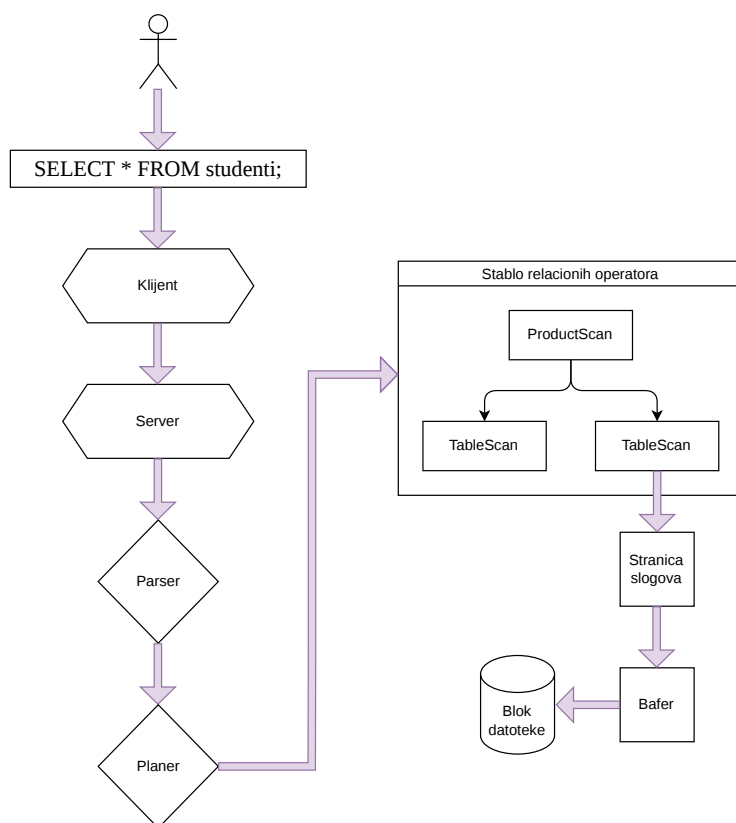
11.2 Ograničenja sistema	69
Spisak slika	71
Spisak listinga	73
Spisak tabela	75
Spisak korišćenih skraćenica	77
Spisak korišćenih pojmova	79
Biografija	81
Literatura	83
Грешке у литературним наводима	85
TODOs	87

Glava 1

Uvod

1.1 O sistemu

LBDB je višekorisniški sistem sa transakcijama za upravljanje relacionim bazama podataka. Njegova svrha je da prima naredbe dobijene od klijenata, intepretira ih po standardu SQL programskog jezika i da vrati rezultat tim klijentima. Rezultat može biti broj pogođenih redova za slučaj modifikacionih operacija ili rezultujuća tabela za slučaj upita. Zasad efikasnog interpretiranja, potrebno je obezbediti algoritme i strukture podataka za sledeće module: upravljanje datotekama, rad sa transakcijama, rad sa metapodacima, parsiranje upita, pravljenje planova izvršavanja upita, izvršavanje upita i za mrežnu komunikaciju. Moduli ovog sistema su raspoređeni tako da se svaki brine o jednoj grupi algoritama i struktura podataka kroz koju upit prolazi.



Slika 1: Uprošćena arhitektura sistema

TODO: pretvoriti ovo u pravi flowchart

Svaka komponenta iz uprošćene arhitekture sistema je posebno detaljno objašnjena u nastavku, a uz te komponente se dodatno objašnjavaju i transakcije, koje su isprepletene kroz ceo sistem. Sličan dijagram koji opisuje celu arhitekturu sistema, ali mnogo detaljnije, se može pronaći ovde. **TODO: citirati detaljan dijagram u pregledu sistema**

Osnovna struktura i algoritmi su izvedeni iz knjige *Database Design And Implementation* [1], a njihova unapređenja su deo ovog rada. Knjiga definiše zadatke na kraju svakog modula i ti zadaci su osnova za unapređivanje sistema. Detaljan spisak urađenih zadataka i njihovih beleški se može pronaći u okviru repozitorijuma¹.

1.2 Klijentsko serverska arhitektura

LBDBServer definiše ulaznu tačku servera. Zahteva port na kome će se server pokrenuti i direktorijum baze podataka. Obrađuje klijente koji se povezuju na njega preko LBDBClient klijentske ulazne tačke.

1.3 Sistem za obradu upita

LBDB klasa predstavlja najviši apstrakcioni nivo sistema obrade upita na koji se server oslanja. Služi za orkestraciju upravljača glavnim podsistemima: [menadžer metapodataka](#), [menadžer transakcija](#) i [planer](#). Takođe, klasa LBDB je odgovorna za inicijalizaciju i oporavljanje sistema od neočekivanog gašenja, za određeni direktorijum baze podataka.

¹<https://github.com/lukascobbler/lbdb>

Glava 2

Upravljanje datotekama

Upravljanje datotekama se vrši kroz više slojeva u sistemu, gde je svaki sloj odgovoran za organizaciju datoteka na različitom apstrakcionom nivou. Osnovna premisa je da sve operacije sa datotekama to jest diskom moraju da se izvršavaju u jedinicama blokova, jer je operativni sistem a i hardver (disk) optimizovan za rad sa njima [1]. Zbog toga što je blok najmanja jedinica interakcije sa diskom i datotekama, sva čitanja, pisanja i modifikacije podataka se rade zajedno sa celim blokom gde se ti podaci nalaze, a ne direktno.

2.1 Upravljanje blokovima

Sistem za upravljanje blokovima je primarno zadužen za dobavljanje bloka sa diska gde se nalaze traženi podaci i za korektno zapisivanje *log* podataka. Svaki blok ima svoj unikatni identifikator, koji je predstavljen Java *record* strukturom. Svaki blok je vezan za datoteku i ima svoju blok poziciju u toj datoteci.

```
public record BlockId(String filename, int blockNum) { }
```

Listing 1: Identifikator bloka

2.1.1 Interfejs ka *file* sistemu operativnog sistema

Najniži nivo apstrakcije predstavlja menadžer datoteka (*FileManager* klasa) koji ima funkciju interfejsa ka *file* sistemu operativnog sistema i nema predstavu šta se nalazi u samim datotekama *LBDB* sistema. Menadžer datoteka čuva pokazivače na sve datoteke kojima sistem upravlja i omogućava višenitni bezbedan pristup istim, upotrebom Java *synchronized* ključne reči. Višenitni bezbedan pristup omogućava sistemu da podrži više različitih klijenata u isto vreme, ali nije dovoljan samo na ovom sloju, već je [detaljno obrađen](#) u okviru transakcija.

Moguće je podesiti sistem da koristi proizvoljnu veličinu jednog bloka, zavisno od prirode podataka kojima će baza podataka biti popunjena i to je glavni [parametar](#) menadžera datoteka.

2.1.2 Stranice

Stranica (eng. *page*) predstavlja sirove bajtove jednog bloka učitane u radnu memoriju. Iako nije nužno potrebno, sve vrednosti iz stranice zajedno sa njihovim tipom se pretvaraju u ekvivalentne Java objekte (koristeći *ByteBuffer* standardnu Java klasu) da bi se omogućio pristup operacijama iz Java standardne biblioteke za te tipove. Kada menadžer datoteka dobavlja određeni blok, bajtovi tog bloka se smeštaju u radnu memoriju u objekat stranice.

2.1 Upravljanje blokovima

Stranice su na apstrakcionom nivou ispod sistema baferovanja i koriste se kao potpora tog sistema.

Sistem podržava sledeće proste i kompozitne tipove podataka: *String*, *Boolean* i *Integer*. Kod prostih tipova, iz stranice se samo čitaju njihovi bajtovi i pretvaraju u određeni Java objekat tog tipa, dok kod kompozitnih tipova kao što je *String* potrebno je znati dužinu, same bajtove i kodiranje da bi se konstruisao Java objekat *String* tipa.

2.1.3 Upravljanje log datotekom

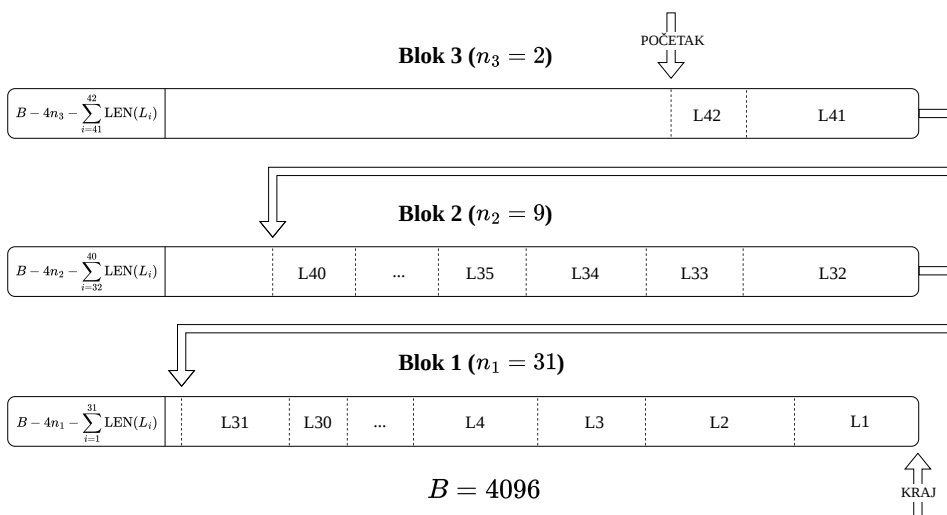
Jedan *log* je niz bajtova čije interpretiranje ukazuje na to kako se promenila neka vrednost u nekom bloku.

Log datoteka (čija je lokacija [podesiva](#)) je specijalna vrsta datoteke u kojoj se čuva niz *log*-ova. Na apstrakcionom nivou upravljanja blokovima, nije bitan sadržaj ove datoteke, već samo algoritmi neophodni za korektno prolaženje kroz nju i algoritmi za dodavanje novih *log*-ova. Ovi algoritmi se nalaze u menadžeru *log*-ova (LogManager klasa) i *log* iteratoru (LogIterator klasa).

Log datoteka je struktuirana tako da se noviji *log*-ovi nalaze u blokovima bližim kraju datoteke, a unutar jednog bloka *log* fajla noviji *log*-ovi se nalaze pri početku bloka. Ako nema mesta da se upiše nov *log*, prelazi se u sledeći blok *log* fajla.

Menadžer *log*-ova obezbeđuje algoritme upravljanja *log* datotekom tako što čuva stranicu poslednjeg bloka *log* fajla. Upravljanje *log* datotekom podrazumeva dodavanje novih *log*-ova, upis *log*-ova na disk i arhiviranje *log* datoteke.

Iterator *log*-ova obezbeđuje čitanje *log* datoteke u korektnom redosledu i implementiran je pomoću Java Iterator interfejsa.



Slika 2: Izgled *log* datoteke i smer iteracije

Svaki blok *log* datoteke na nultoj poziciji sadrži četvorobajtni broj koji predstavlja lokaciju prvog *log*-a u tom bloku. Svaki *log* se sastoji iz sadržaja i svoje dužine koja je isto četvorobajtni broj.

Prilikom dodavanja novog *log*-a, sistem izračunava *log* sekvencu novog *log*-a (eng. *log sequence number*) koja unikatno identifikuje zapisan *log* i može se koristiti u daljim podsistemima za forsiranje pisanja prethodnih *log*-ova u *log* fajl i time garantovati redosled operacija.

2.2 Upravljanje baferima

Čišćenje stranice iz memorije nakon završetka operacije koja ju je koristila može biti jako neefikasno jer se za ponovni pristup tom bloku mora odlaziti do diska, pogotovo u višekorisničkim kontekstima i situacijama kada se istim blokovima često pristupa. Zbog toga se uvodi koncept bafera (eng. *buffer*) koji, uz algoritme u menadžeru bafera (*BufferManager* klasa), omogućava stranicama da ostanu u memoriji prilagodljivu količinu vremena. Posledica ovog sistema je da direktan pristup podacima sa diska više nije moguć, već se sve operacije obavljaju kroz bafere.

Bafer je omotač oko stranice koji sadrži dodatne podatke koji se mogu iskoristiti za implementaciju raznih algoritama ubrzanja sistema:

- kada je učitán
- kada je bilo poslednje pristupanje
- koji je njegov redni broj u listi bafera
- koja transakcija ga je poslednji put modifikovala i *log* sekvencu njene poslednje operacije
- koliko transakcija ga trenutno upotrebljavaju, kraće rečeno broj pinova

Pošto je količina radne memorije ograničena, i količina bafera u sistemu isto mora biti ograničena. Menadžer bafera ima listu (**odredene veličine**) bafera sa kojima raspolaže i potrebno je da poveća stepen iskorišćenosti bafera iz te liste što je više moguće. U idealnom, hipotetičkom scenariju, menadžer bafera bi predvideo budućnost i znao kojim baferima bi se sledeće pristupalo i izbacio iz memorije one koji su vremenski najdalje od pristupa. Ovaj scenario je očigledno nemoguć, pa je potrebno iskoristiti algoritme koji najbolje “predviđaju budućnost” na osnovu realnih podataka. Kada se bafer izbaci iz memorije, njegov sadržaj se piše u blok za koji je vezan i na taj način se podaci perzistiraju. Postoje i **druge situacije** kada se sadržaj bafera odmah zapisuje na disk.

Da bi se bafer izbacio iz memorije, ne sme da bude deo ni jedne aktuelne transakcije, to jest broj pinova mu mora biti nula. Postoje dva scenarija kada stranica koja do sad nije bila u memoriji treba da se mapira na bafer:

- U slučaju da ne postoji ni jedan bafer sa nula pinova, nova stranica čeka određeni vremenski period da se oslobodi neki bafer i ako se ni jedan bafer ne oslobodi, vraća se greška klijentu.

- U slučaju da postoji više bafera sa nula pinova, potrebno je izabrati koji će biti izbačen iz radne memorije pomoću algoritma izbora.

2.2.1 Algoritmi izbora smene bafera

U optičaju je nekoliko algoritama [1] za izbor bafera koji će biti smenjen i opcije su prikazane u okviru `BufferStrategy` enumeracije ([podesivo](#)):

```
public enum BufferStrategy {  
    NAIVE, FIFO, LRU, CLOCK,  
    FIRST_UNMODIFIED, LRM  
}
```

Listing 2: Razičiti algoritmi izbora smene bafera

2.2.1.1 Naive

Naivni algoritam, kako mu i ime kaže, ne razmišlja mnogo o baferu kojeg će smeniti već samo uzima prvi bafer koji ima nula pinova. Očekivane loše performanse jer je velika šansa da će se smeniti bafer koji će se uskoro opet koristiti. **TODO: citirati performanse svih alg**

2.2.1.2 FIFO

FIFO (eng. *first in first out*) algoritam smenjuje bafer koji je najranije ušao u listu bafera. Performanse su bolje od naivnog algoritma ali *FIFO* algoritam pati od toga da će zameniti i jako često korišćene bafere iako su najranije ušli u sistem, na primer baferi gde se čuvaju blokovi metapodataka sistema.

2.2.1.3 LRU

LRU (eng. *least recently used*) algoritam smenjuje bafer koji je najdavnije korišćen. Performanse su odlične jer ako bafer dugo nije korišćen, verovatno se neće još dugo ni koristiti.

2.2.1.4 Clock

Clock algoritam smenjuje prvi bafer koji ima nula pinova, ali pretragu počinje od prethodnog smenjenog bafera, formirajući krug ili sat. Performansa je okej jer je šansa da je bitan bafer pinovan velika, pa se on preskače kada se prolazi kroz krug.

2.2.1.5 First unmodified

First unmodified algoritam smenjuje prvi bafer koji pronade da nije modifikovan i da ima nula pinova ili ako je svaki modifikovan, prvi koji ima nula pinova. Performansa može biti bolja od naivnog algoritma, ali može se desiti da modifikovan bafer neće dugo biti korišćen pa je onda to bacanje bafera.

2.2.1.6 LRM

LRM (eng. *least recently modified*) algoritam smenjuje bafer koji ima nula pinova i koji je poslednje izmenjen, to jest bafer sa najmanjim brojem *log* sekvence. Predpostavka je da

modifikovani bafer neće ponovo biti korišćen neko vreme jer je transakcija već završila. Performansa deluje okej, ali je algoritam dosta nepredvidiv.

2.3 Slogovi

Podsistem za upravljanje baferima je generalan i ne pruža nikakvu strukturu podataka unutar samih bafera, to jest blokova. Na nivou relacione baze podataka, najmanja jedinica interakcije nije jedan blok, već jedan slog neke tabele i potrebno je blokove organizovati tako da se to omogući.

2.3.1 Struktuiranje

Da bi se podržalo kreiranje perzistentne strukture jednog sloga nove tabele, potrebno je definisati mehanizme kojima će klijenti opisivati tu strukturu. Na apstrakcionom nivou upravljanja datotekama, sistem se samo brine o tome da je teoretska i fizička struktura ispoštovana, a perzistiranje i samo kreiranje strukture je zadatak viših podsistema.

2.3.1.1 Šema

Svaki slog jedne tabele se sastoji od istih metapodataka, to jest istih kolona. Svaka kolona se opisuje svojim tipom, svojom dužinom na disku i tome da li može sadržati *NULL* vrednosti.

```
public record FieldInfo(  
    DatabaseType type, int runtimeLength, boolean nullable  
) { }
```

Listing 3: Opis jedne kolone

Svaki od tipova je definisan u SQL Java standardnoj biblioteci, ali korišćenje tih vrednosti direktno može dovesti do nekompletnosti na raznim mestima gde su tipovi korišćeni u sistemu, pa je zbog toga uvedena enumeracija koja striktno definiše podržane tipove, zajedno sa njihovom podrazumevanom dužinom u bajtovima. Tip *VARCHAR*, to jest *String* nema podrazumevanu dužinu jer je različita za svako polje. Dodatno postoji i *NULL* tip koji označava nemanje vrednosti za to polje.

```
import java.sql.Types;  
public enum DatabaseType {  
    INT(Types.INTEGER, 4),  
    BOOLEAN(Types.BOOLEAN, 1),  
    VARCHAR(Types.VARCHAR, -1),  
    NULL(Types.NULL, 0);  
  
    public final int sqlType, length;  
    DatabaseType(int sqlType, int length) {  
        this.sqlType = sqlType; this.length = length;  
    }  
}
```

Listing 4: Podržani tipovi kolona

Šema jedne tabele je skup podataka o kolonama te tabele zajedno sa imenima tih kolona. Bitno je napomenuti da tabele u svojoj osnovnoj definiciji predstavljaju podatke koje se nalaze u datotekama, ali to nije uvek slučaj. Postoje i virtuelne tabele koje su rezultati upita i mogu, ali ne moraju da se direktno mapiraju na tabele koje se nalaze u datotekama. Moguće je kombinovati više tabela u jednu tabelu i **dodati virtuelne kolone** koje se ne nalaze u datoteci već su njihove vrednosti izvedene na osnovu neke kalkulacije.

2.3.1.2 Raspored polja

Šema predstavlja teoretski izgled jedne tabele, ali to nije dovoljno da bi se taj izgled perzistirao i mogao ponovo rekreirati. Zbog toga je potrebno uvesti mehanizam pamćenja i fizičkih karakteristika kolona tabele (postoji samo za nevirtuelne tabele). Taj mehanizam se realizuje preko rasporeda polja (eng. *layout*).

```
public class Layout {  
    private final Schema schema;  
    private final Map<String, Integer> offsets;  
    private final Map<String, Integer> fieldPositions;  
    private final int recordSize;  
}
```

Listing 5: Fizičke karakteristike polja

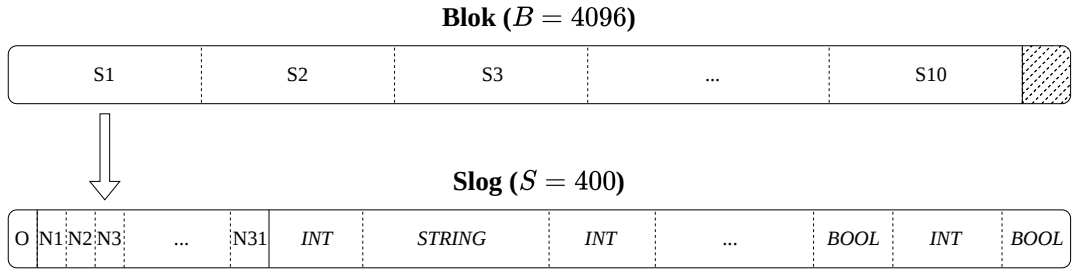
Za svaku kolonu postoji se pamte sledeće fizičke karakteristike: pozicija početka vrednosti te kolone, maksimalna dužina vrednosti te kolone i pozicija te kolone u šemi. Takođe, pamti se i celokupna dužina celog sloga. Kolone se identifikuju pomoću njihovog naziva.

2.4 Primena strukture na blok

Nakon definisanja fizičke strukture sloga tabele, potrebno je primeniti tu fizičku strukturu na blokove datoteka. U LBDB sistemu, jedan blok sadrži fiksni broj slogova koji su svi iz iste tabele i ne postoje vrednosti promenjive dužine. Ovo je jedna od glavnih ograničenja sistema. **TODO: citirati zaključak sa limitacijom sistema, fiksni nespanovani slogovi**

Pošto su svi slogovi iste dužine, $\frac{B}{S}$ slogova staje u jedan blok, gde B predstavlja dužinu bloka u sistemu, S predstavlja dužinu jednog sloga te tabele, a $B - S * \lfloor \frac{B}{S} \rfloor$ prostora ostaje neiskorišćeno (sve vrednosti su u bajtovima). Slogovi u blokovima čuvaju samo vrednosti kolona, ali ne i metapodatke tih kolona. Podsystem upravljanja datotekama se ne brine o metapodacima kolona, već za to postoji **poseban podsystem** koji se nadograđuje na ovaj.

Ipak, u okviru jednog sloga se čuvaju metapodaci o tome koje vrednosti nisu prisutne, to jest imaju *NULL* vrednost i to da li je slog obrisan. Rezerviše se četvorobajtno zaglavlje na početku svakog sloga i njegovi bitovi predstavljaju ove metapodatke. Da li je slog označen kao obrisan se predstavlja prvim bitom (O), dok ostalih 31 bitova (N_i) označavaju da li polje na toj poziciji ima *NULL* vrednost. Zbog ovoga postoji ograničenje na broj polja po tabeli, maksimalno 31 polje.



Slika 3: Izgled jednog bloka struktuiranog sa slogovima

2.4.1 Stranica slogova

RecordPage klasa enkapsulira svu logiku održavanja strukture individualnog bloka tako što pruža interfejs za postavljanje vrednosti samo na osnovu imena kolone i broja sloga u tom bloku. Takođe, pruža interfejs za postavljanje *NULL* vrednosti i pretragu slobodnih ili zauzetih slogova u bloku za koji je povezana. Za pristup svim blokovima jedne tabele, potrebno je sukcesivno konstruisati objekte RecordPage klase, što je posao podsistema [relacionih operatora](#).

Bitno je napomenuti da logika stranice slogova za postavljanje vrednosti ne radi sâmo postavljanje vrednosti, već samo računa gde ta vrednost treba biti postavljena. Postavljanje vrednosti [delegira](#) sistemu transakcija.

Kao što za blokove postoji [unikatni identifikator](#), tako unikatni identifikator postoji i za slogove i sadrži u kom je bloku slog i na kojoj je poziciji unutar bloka:

```
public record RecordId(int blockNum, int record) { }
```

Listing 6: Identifikator sloga

Identifikatori slogova se koriste za direktan skok na neki slog.

Glava 3

Upravljanje transakcijama

Transakcije su osnovni mehanizam za garantovanje različitih osobina otpornosti sistema za upravljanje relacionim bazama podataka. Omogućavaju sistemu da vrši atomične operacije, da se uvek održi u konzistentnom stanju, da izoluje nezavisne konkurentne operacije i da garantuje perzistentnost podataka. Generalno, ove osobine se zovu *ACID* osobine (eng. *atomicity, consistency, isolation, durability*) [2].

Svaka operacija u sistemu mora biti izvršena u okviru jedne transakcije, ali se jedna transakcija može sastojati i od više operacija koje se sve moraju ili uspešno izvršiti ili se moraju sve poništiti. Svaka transakcija ima početak i kraj. Kraj može biti potvrda (eng. *commit*) ili poništavanje (eng. *rollback*). Ako je transakcija uspešno potvrđena, od tog trenutka pa na dalje sve izvršene promene moraju biti odmah vidljive drugim transakcijama.

3.1 Realizacija transakcionih mehanizama u sistemu

Vrednost je niz bajtova (određenog tipa), koja dobija semantiku značaj tek na apstrakcionim nivoima iznad nivoa transakcija, naime na nivou struktuiranja blokova u slogove. Na nivou transakcija, vrednosti nemaju semantičko značenje, ali da bi sistem obezbedio visok stepen usklađenosti sa *ACID* osobinama, operacije nad vrednostima se obavljaju isključivo kroz transakcije koje enkapsuliraju svu potrebnu logiku tih osobina.

3.1.1 Transakcije kao glavno mesto pristupa vrednostima

Na nivou slogova, operacije se izvršavaju u celinama jednog sloga, ali gradivni elementi tih slogova su ipak vrednosti. Stranica slogova upravlja pozicijama vrednosti, a sam pristup njima delegira transakciji za koju je vezana.

Pošto blokovi gde se vrednosti nalaze moraju biti u memoriji da bi se njima pristupalo, potrebno je učitati ih u sistemske bafere. Podsistem za upravljanje baferima je svestan samo načina mapiranja blokova na bafere, ali nema nikakve garancije o redosledu pristupa, ne čuva od konfliktujućih operacija i ne prati istoriju izmena. To je posao transakcija.

Svi baferi u radnoj memoriji sistema se grupišu u transakcije, tako što svaka transakcija čuva listu svojih bafera i radi operacije samo nad njima. Bafer ulazi u listu neke transakcije tako što ga transakcija pinuje, i time označava da se sadržaj tog bafera ne sme vratiti na disk dok transakcija ne završi sa njim (dok ga ne unpinuje). Stranica slogova i transakcija rade u paru: stranica slogova zna kojem bloku želi da pristupi, a transakcija zna kako da obezbedi da operacije nad baferom tog bloka ispoštuju *ACID* osobine.

3.1.2 Sistem za oporavak

Sistem za oporavak je podsistem u okviru transakcija koji sadrži prvi deo logike zbog kojeg se sav pristup vrednostima radi kroz transakcije. Primarno se brine o oporavku sistema, ali se brine i o poništavanju operacija jedne transakcije (eng. *rollback*). Obuhvata ACD (eng. *atomicity, consistency, durability*) osobine.

LBDB sistem je softversko rešenje koje radi u kontekstu nekog hardvera i operativnog sistema. Svaki od slojeva na koje se LBDB sistem oslanja, ima mogućnost da zakaže zbog nekog faktora koji je izvan kontrole LBDB sistema. Primeri su nestajanje struje, ubijanje LBDB serverskog procesa ili neuspešno pisanje na disk. Ako se u trenutku zakazivanja izvršava neka operacija koja menja podatke u sistemu, ti podaci će biti izgubljeni, a sistem će biti ostavljen u nekonzistentnom stanju. Korišćenje sistema koji je u nekonzistentnom stanju može dovesti do lošeg tumačenja podataka, ali može biti i opasno u pojedinim situacijama. Zbog ovoga se uvodi podsistem koji oporavlja sistem od nekonzistentnog stanja.

Konzistentno stanje se može definisati sa sledeće dve osobine [1], [2]:

- sve nedovršene transakcije su poništene,
- sve modifikacije potvrđenih (eng. *committed*) transakcija moraju biti na disku

3.1.2.1 Sistem *log*-ovanja

Log podaci su u *log* fajlu uvek poređani redom kojim su se izvršili u okviru jedne transakcije. Pisanjem *log* podataka se postiže visoko granulirana istorija izmena. Osnova svakog algoritma oporavka je prolaženje kroz istoriju izmena počevši od kraja, da bi se te izmene poništile ili ponovo primenile tačnim redosledom.

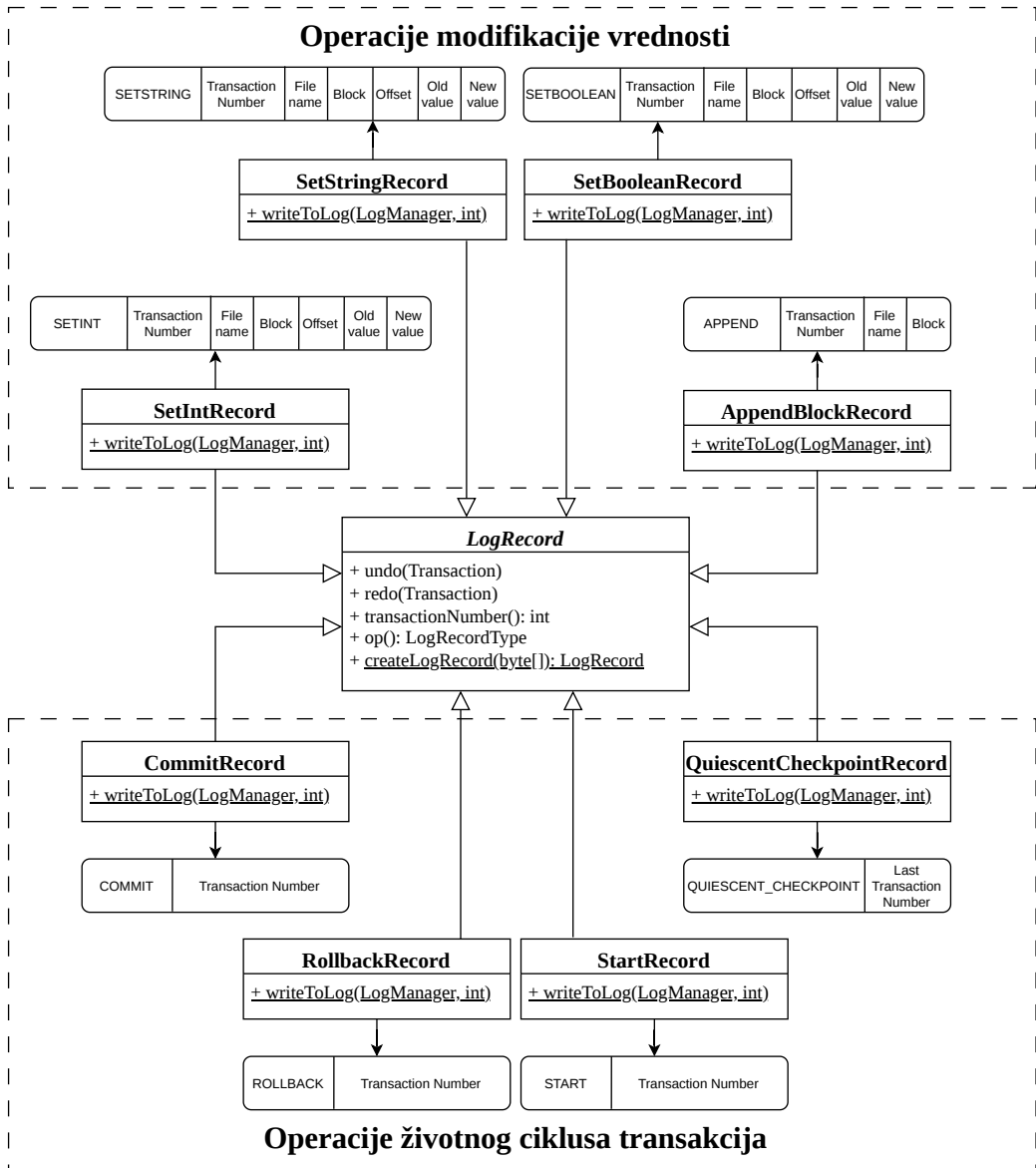
U sistemu postoji dve glavne grupe operacija: operacije modifikacije vrednosti i operacije životnog ciklusa transakcija. Za svaku operaciju se definiše struktura *log*-a koja opisuje tu operaciju. Za operacije modifikacije vrednosti, u *log*-u stoje svi podaci neophodni da se ta operacija poništi ili ponovo primeni, dok u *log*-ovima operacija životnog ciklusa transakcija stoje svi neophodni podaci da se prati rad transakcija.

Pošto u *log*-ovima operacija modifikacije vrednosti uvek stoje i stara i nova vrednost, poništavanje ili ponova primena operacije je trivijalna. Specijalna operacija modifikacije je operacija umetanja novog bloka na kraju datoteke, koja ne menja nikakvu vrednost ali je i dalje potrebno pratiti njene pozive zbog održavanja korektne veličine datoteka. Algoritam poništavanja umetanja novog bloka nije trivijalan jer nije samo zamena vrednosti, već je potrebno označiti bafer tog bloka kao nemodifikovan i skratiti datoteku za jedan blok. U *log* datoteku se uvek zapisuje nov *log* za pozvanu operaciju modifikacije pre same obrade te operacije.

Operacije životnog ciklusa transakcija je potrebno pratiti da bi sistem oporavke znao koje transakcije su se uspešno i neuspešno izvršile i na osnovu toga reagovati na operacije

modifikacije. Marker kontrolne tačke je specijalni zapis koji označava kada nije potrebno dalje prolaziti kroz *log* datoteku. Više o njemu u [algoritmima oporavke sistema](#).

Hijerarhija *log* zapisa počinje od glavnog interfejsa *LogRecord* koji definiše neophodne operacije koje će zvati algoritmi oporavke sistema. Operacije životnog ciklusa transakcija ostavljaju praznu implementaciju *undo* i *redo* metoda jer ne modifikuju podatke. Dodatno, svaki tip *log* zapisa ima prateću statičku metodu *writeToLog* koja enkapsulira logiku pisanja same strukture konkretnog *log*-a preko menadžera *log*-ova za neki identifikator transakcije.



Slika 4: Operacije, njihovi *log* tipovi i struktura *log* tipova

3.1.2.2 Algoritmi oporavka sistema

Algoritam oporavka sistema je procedura koja prati korake definisane strategijom oporavka koju sistem koristi i zajedno sa podacima iz *log* datoteke vraća sistem u konzistentno stanje. Svaki algoritam oporavka mora biti idempotentan, jer sistem može naglo prestati sa radom i dok je u procesu oporavljanja [1]. Proces oporavka se vrši u okviru podizanja sistema.

Postoje tri generalna algoritma oporavke [1]:

- ponovna primena uspešnih transakcija i poništavanje neuspešnih transakcija (eng. *undo redo recovery*),
- samo poništavanje neuspešnih transakcija (eng. *undo only recovery*),
- samo ponovna primena uspešnih transakcija (eng. *redo only recovery*).

Izbor algoritma oporavke utiče na to kada će sadržaj bafera biti upisan na disk.

U LBDB sistemu, implementirani su *undo redo* i *undo only* algoritmi oporavke i moguće je postaviti koji će sistem koristiti u okviru [sistemskih podešavanja](#).

3.1.2.2.1 Undo redo recovery

Algoritam koji radi oba dela operacije ne zahteva dodatne restrikcije na redosled upisa sadržaja bafera na disk. Algoritam se deli na dva koraka: faza poništavanja i faza ponovne primene.

Faza poništavanja (eng. *undo*):

- Za svaki log zapis (čitajući unazad od kraja *log-a*):
 - ▶ Ako je tekući zapis *commit* zapis, dodaj tu transakciju u listu potvrđenih transakcija.
 - ▶ Ako je tekući zapis *rollback* zapis, dodaj tu transakciju u listu poništenih transakcija.
 - ▶ Ako je tekući zapis *update* zapis (modifikaciona operacija), i transakcija nije ni u listi potvrđenih ni u listi poništenih, vrati staru vrednost na zadatoj lokaciji.

Faza ponovne primene (eng. *redo*):

- Za svaki log zapis (čitajući unapred od početka *log-a*):
 - ▶ Ako je tekući zapis *update* zapis i transakcija je u listi potvrđenih transakcija, upiši novu vrednost na zadatoj lokaciji.

Glavna prednost ovog algoritma je to što ne utiče na redosled pisanja bafera na disk, ali mana je to što je proces oporavka sporiji. Ako se pretpostavi da su iznenadna gašenja sistema retka, prednosti ovog algoritma pobeđuju ostale algoritme.

3.1.2.2.2 Undo only recovery

Undo only recovery algoritam radi samo fazu poništavanja jer je siguran da su svi baferi *committed* transakcija već zapisani na disku. Ovo se postiže modifikacijom *commit* algoritma tako da se prvo upisuje sadržaj bafera na disk pre pisanja *commit log-a*. Glavna prednost

ovog algoritma je brzina, jer se kroz *log* fajl prolazi samo jednom, ali mana je to što *commit* operacija postaje mnogo sporija.

3.1.2.2.3 *Redo only recovery*

Redo only recovery algoritam radi samo fazu ponovne primene jer je siguran da baferi transakcija koje nisu završile sigurno nisu zapisani na disk. Ovo se postiže modifikacijom transakcija tako da su baferi u njihovim listama pinovani dokle god se transakcija ne završi. Glavna prednost ovog algoritma je brzina jer se kroz *log* fajl prolazi samo jednom, ali mana je to što baferi ostaju pinovani mnogo duže, drastično usporavajući sistem.

3.1.2.2.4 *Mirna kontrolna tačka*

Što se sistem duže koristi, to će njegov *log* fajl postajati obimniji jer uvek sadrži kompletnu istoriju izmena podataka. Prolazak kroz ceo *log* fajl na prilikom svakog pokretanja sistema može trošiti nepotrebno mnogo resursa, a u jednom trenutku će i biti nemoguće.

Trenutak kada algoritam oporavka ne mora da čita *log* fajl dublje se može okarakterisati sa dve osobine [1]:

- svi prethodni *log* zapisi su napisani od završenih transakcija (*undo* faza algoritma)
- baferi tih transakcija su napisani na disk (*redo* faza algoritma)

Kontrolna tačka predstavlja zapis u *log* fajlu posle kojeg se ne mora čitati dalje jer je sistem provereno potvrdio da obe osobine važe. Sistem trivijalno ovo može da obezbedi nakon što je završio sa oporavkom, a algoritam koji ovo obezbeđuje u ostalim situacijama se može pronaći [ovde](#). LBDB implementira samo logiku za mirnu kontrolnu tačku (eng. *quiescent checkpoint*), ali postoji i nemirna kontrolna tačka (eng. *nonquiescent checkpoint*).

Nakon oporavka, umesto pisanja kontrolne tačke, sistem radi arhiviranje *log* datoteke. Arhiviranje je ekvivalentna operacija, a omogućava preglednije održavanje sistema.

3.1.2.3 Algoritam poništavanja transakcije

Sistem radi poništavanje transakcije tako što prolazi kroz *log* datoteku od kraja ka početku i poništava svaku modifikacionu operaciju te transakcije, a staje sa prolaskom *log* datoteke kada naiđe na operaciju koja označava početak te transakcije.

3.1.3 Bezbedan višenitni pristup

Sistem za bezbedan višenitni pristup je podsistem u okviru transakcija koji sadrži drugi deo logike zbog kojeg se sav pristup vrednostima radi kroz transakcije. Primarno se brine o rešavanju konflikta konkurentnog pristupa istim blokovima. Obuhvata *I* (eng. *isolation*) osobinu.

Priroda višenitnih pristupa uvodi nedeterminističan redosled operacija koji implicira konflikte. Konflikt je kada rezultat dve operacije zavisi od njihovog redosleda izvršavanja. Postoje dve vrste konflikta: *write-write* konflikt i *read-write* konflikt. Kod *write-write*

konflikta, jedna operacija modifikuje neku vrednost, pa druga modifikuje istu vrednost. Kod *read-write* konflikta, jedna operacija čita neku vrednost, a druga modifikuje istu tu vrednost. Konflikti ne mogu da se dese kod *read-read* operacija ili ako operacije rade nad vrednostima koje se nalaze u različitim blokovima [1].

Sprečavanje konfliktujućih operacija u sistemu se postiže preko sistema katanaca, koji omogućavaju korektan redosled pristupa vrednostima za čitanje i pisanje. Protokol zaključavanja u LBDB sistemu definiše sledeća pravila rada sa katancima:

- pre čitanja vrednosti iz bloka, potrebno je steći *deljeni* katanac za taj blok,
- pre pisanja vrednosti u blok, potrebno je steći *ekskluzivni* katanac za taj blok,
- nakon završetka transakcije, potrebno je pustiti sve katance za sve blokove koje je ta transakcija zaključala.

Poštovanje ovakvog protokola zaključavanja uvek garantuje tačnost rada sa vrednostima, ali znatno smanjuje konkurentnost sistema. Povećanje konkurentnosti sistema se radi izborom izolacionog nivoa individualnih transakcija. Izolacioni nivoi transakcija povećavaju konkurentnost ali žrtvuju tačnost pročitanih vrednosti tako što upravljaju životnim ciklusom *deljenih* katanca na različite načine [1].

Tabela 1: Različiti izolacioni nivoi transakcija

Izolacioni nivo	Problemi	Puštanje <i>deljenih</i> katanaca	<i>EOF</i> marker
Serijalizujući (eng. <i>serializable</i>)	ne postoje	<i>deljeni</i> katanci se drže do završetka transakcije	postoji <i>deljeni</i> katanac
Ponovno čitanje (eng. <i>repeatable read</i>)	fantomske vrednosti	<i>deljeni</i> katanci se drže do završetka transakcije	ne postoji <i>deljeni</i> katanac
Čitanje samo potvrđenih vrednosti (eng. <i>read committed</i>)	fantomske vrednosti, promenjene vrednosti	<i>deljeni</i> katanci se puštaju odmah nakon čitanja	ne postoji <i>deljeni</i> katanac
Čitanje i nepotvrđenih vrednosti (eng. <i>read uncommitted</i>)	fantomske vrednosti, promenjene vrednosti, "prljava" čitanja	ne koriste se <i>deljeni</i> katanci	ne postoji <i>deljeni</i> katanac

Različiti izolacioni nivoi se mogu implementirati i preko *MVCC* (eng. *multi version concurrency control*) pristupa [3], ali on nije podržan u okviru LBDB sistema.

Izolacioni nivoi transakcija su koncipirani tako da svaki nivo izolacije rešava sve probleme nivoa ispod njega.

- Problem “prljavog” čitanja je kada transakcija *A* može da čita vrednosti koja je transakcija *B* izmenila, pre nego što je transakcija *B* završila.
- Problem promenljivih vrednosti je kada transakcija *A* pročita neku vrednost, transakcija *B* je modifikuje, transakcija *A* ponovo pročita istu vrednost i dobije vrednost koju je transakcija *B* modifikovala umesto vrednosti koju je transakcija *A* prvobitno pročitala.
- Problem fantomskih vrednosti je kada transakcija *A* pročita sve vrednosti nekog blokovskog opsega, transakcija *B* doda novu vrednost i proširi taj blokovski opseg sa novim blokom, i umesto da transakcija *A* pri ponovnom čitanju svih vrednosti dobije istu listu prvobitnih vrednosti, dobije i novu vrednost iz novog bloka koju je transakcija *B* dodala.

Pošto je nivo granularnosti zaključavanja na nivou jednog bloka, fantomska čitanja se mogu desiti samo ako se na kraju neke datoteke doda nov blok. Sprečavanje ovoga se dešava specijalnim *deljenim* katancom nad *EOF* markerom. *EOF* marker glumi blok koji tek treba da se doda na kraju neke datoteke, ali koji fizički ne postoji u datoteci.

Pomenuti izolacioni nivoa transakcija se odnose samo na operacije koje čitaju vrednosti. Operacije koje modifikuju vrednosti uvek moraju poštovati korektno dobijanje *ekskluzivnih* katanaca. Transakcije na individualnom nivou mogu tolerisati neprecizne podatke, ali kada bi se dobijanje *ekskluzivnih* zaobišlo, cela baza podataka bi postala neupotrebljiva.

Podsystem bezbednog višenitnog pristupa LBDB sistema implementira samo serijalizujući izolacioni nivo transakcija.

3.1.3.1 Katalog katanaca

Instanca klase *LockTable* je deljena za sve transakcije u sistemu. U njoj se realizuju mehanizmi praćenja postojanja katanaca za sve blokove.

Deljeni katanac za neki blok je moguće steći bez obzira na postojanje drugih *deljenih* katanaca, ali *ekskluzivni* katanac za neki blok je moguće steći samo kada ne postoji ni jedan drugi katanac bilo koje vrste.

Ako se desi da transakcija nije uspela da zaključa željeni blok (bilo sa *deljenim* ili *ekskluzivnim* katancom) posle određenog vremenskog perioda, se vraća greška klijentu.

3.1.3.2 Upravljanje katancima na nivou individualnih transakcija

Menadžer konkurentnosti (*ConcurrencyManager* klasa) sadrži skup jednostavnih algoritama koji interaguju sa katalogom katanca da bi transakcijama na individualnom nivou omogućili upravljanje katancima. Svaka transakcija ima svoj menadžer konkurentnosti.

3.2 Transakcije i klijenti

Pošto svaka operacija u sistemu mora biti izvršena u okviru neke transakcije, klijentima LBDB sistema je potrebno dodeliti korektne transakcione objekte. Menadžer transakcija upravlja životnim ciklusom transakcija i vezuje ih za klijente. Menadžer transakcija je jedan od tri glavna podsistema LBDB sistema [obrade upita](#).

Nova transakcija počinje čim se prethodna transakcija za koju je klijent vezan završi. Prva transakcija se dodeljuje klijentu prilikom njegovog [povezivanja na sistem](#). Podrazumevani režim završetaka transakcija je automatsko završavanje (eng. *autocommit*) u kom se svaka transakcija sastoji od tačno jedne operacije. Drugi režim završetaka transakcija je manuelni režim, u kom se transakcija može sastojati od više operacija, i u tom slučaju kraj transakcije označava operacija potvrde ili poništavanja koja se dobija od klijenta.

Da bi se postiglo perzistiranje istog transakcionog objekta kroz više operacija, potrebno je pratiti sve aktuelne transakcione objekte u sistemu i unikatno definisati svakog klijenta pa uvek izvršavati operacije u okviru njegovog transakcionog objekta dokle god transakciji ne dođe kraj. Unikadni identifikator klijenta predstavlja njegova sesija. Na apstrakcionom nivou upravljanja transakcijama, sesija nije ništa više nego broj, ali na višim slojevima je potrebno generisati tu sesiju korektno. Detalji generisanja sesije se mogu pronaći [ovde](#).

3.2.1 Algoritam zapisa mirne kontrolne tačke

Da bi mirna kontrolna tačka bila zapisana, ni jedna transakcija ne sme biti aktivna u sistemu, a ovo je evidentno zbog prve osobine mirne kontrolne tačke. Korektan zapis mirne kontrolne tačke se u okviru menadžera transakcija izvršava sledećim algoritmom [\[1\]](#):

- prestaje da prihvata nove transakcije od klijenata
- čeka da svi klijenti završe sa svojom transakcijom, bilo ona manuelna ili automatska
- zapisuje sve bafere na disk
- dodaje zapis mirne kontrolne tačke u *log* datoteku
- ponovo prihvata nove transakcije od klijenata

Menadžer transakcija je jedini koji je svestan činjenice da li su sve transakcije završene, jer ih prati, pa je algoritam zapisa mirne kontrolne tačke implementiran u okviru njega.

Glava 4

Metapodaci

Metapodaci su podaci koji opisuju druge podatke. Iako su podaci struktuirani u okviru slogova (glava 2), sistem im ne može pristupiti ako se ne pobrine o perzistiranju te strukture. Praćenje distribucije vrednosti je korisno prilikom pravljenja efikasnog načina dobavljanja slogova. Podaci koji definišu strukturu slogova i podaci o distribuciji vrednosti su primeri metapodataka kojima sistem barata.

4.1 Kataloške tabelle

Metapodatake koje sistem čuva da bi omogućio rad sa tabelama su podaci o postojećim tabelama i kako kolone tih tabela izgledaju. Ti podaci se čuvaju u okviru sistemskih tabela i one se nazivaju kataloške tabelle. Kataloška tabela *tablecatalog* čuva podatke o postojećim tabelama, dok kataloška tabela *fieldcatalog* čuva podatke o fizičkoj strukturi sloga neke tabelle. Vrednosti ovih tabela su pohranjene iz [rasporeda polja](#) i [šeme](#) koju on sadrži.

Tabela 2: Šema *tablecatalog* tabelle

Kolona	Tip	Opis
<i>tableid</i>	<i>Integer</i>	unikatni identifikator tabelle
<i>tablename</i>	<i>String</i>	ime tabelle
<i>slotsize</i>	<i>Integer</i>	veličina jednog sloga te tabelle u bajtovima

Tabela 3: Šema *fieldcatalog* tabelle

Kolona	Tip	Opis
<i>type</i>	<i>Integer</i>	tip kolone zapisan kao numerička vrednost
<i>runtime length</i>	<i>Integer</i>	maksimalna fizička dužina vrednosti ovog polja
<i>offset</i>	<i>Integer</i>	pozicija vrednosti te kolone
<i>tableid</i>	<i>Integer</i>	unikatni identifikator tabelle u kojoj se kolona nalazi
<i>fieldname</i>	<i>String</i>	ime kolone
<i>nullable</i>	<i>Boolean</i>	da li vrednosti kolone mogu biti <i>NULL</i> vrednost

Kataloške tabele se kreiraju prilikom inicijalizacije sistema. Bitno je napomenuti da se kataloške tabele perzistiraju na isti način kao i sve ostale tabele u sistemu, što znači da će one sadržati i slogove koje opisuju njih same. Time što se kataloške tabele perzistiraju isto kao i korisničke, sistemskim tabelama se može pristupiti putem standardnih mehanizama [relacionih operatora](#).

Svi identifikatori u sistemu (imena kolona, tabela, ...) se implicitno konvertuju tako da sadrže samo mala slova.

4.2 Statistički podaci

Pristup istim slogovima tabela se često može izvršiti na više različitih načina, ali neki načini mogu biti znatno manje efikasni od ostalih. Apstrakcioni nivo upravljanja metapodacima je dužan da obezbedi statističke metapodatke koji pomažu pri proceni vremena izvršavanja određenih načina pristupa. Sâm posao konstruisanja efikasnog načina pristupa je briga [podsistema planiranja](#).

Statistički metapodaci neke tabele uključuju:

- broj blokova tabele
- broj slogova u tabeli
- broj različitih vrednosti kolona tabele
- broj *NULL* vrednosti kolona tabele

4.2.1 Računanje statističkih podataka

Prilikom inicijalizacije sistema, računaju se statistički metapodaci za svaku tabelu u sistemu, a svakih 100 poziva dobavljanja metapodataka za bilo koju tabelu se osvežavaju statistički metapodaci za sve tabele. Ovaj način osvežavanja nije idealan jer pauzira sistem dok se računanje statističkih metapodataka ne završi. **TODO: citirati zaključak za limitaciju sistema racunanja statistickih podataka**

Broj blokova tabele i broj slogova u tabeli se trivijalno dobijaju iteracijom kroz svaki slog.

Broj različitih vrednosti kolone tabele nije moguće izračunati precizno, jer je za to potrebno čuvanje svih jedinstvenih vrednosti te kolone u radnoj memoriji. Male nepreciznosti neće uticati na procenu vremena izvršavanja operacija, pa je iskorišćena probabilistička struktura podataka *HyperLogLog* [4] koja rešava *count distinct* problem i ona ne čuva sve jedinstvene vrednosti u radnoj memoriji. Jedna takva struktura se dodeljuje za svaku kolonu.

Brojanje *NULL* vrednosti kolona tabele se svodi na čuvanje prostog brojača za svaku kolonu.

4.3 Pristup metapodacima

Menadžer metapodataka je glavno mesto pristupa svim ostalim metapodacima. Sastoji se iz menadžera metapodataka tabela i menadžera statističkih metapodataka. Menadžer metapodataka je jedan od tri glavna podsistema LBDB sistema [obrade upita](#).

Glava 5

Relacioni operatori

SQL programski jezik je jezik deklarativnog tipa. To znači da se preko njega specificira šta treba da se uradi sa podacima (dobavljanje, filtriranje, modifikacija, ...), ali za razliku od proceduralnih programskih jezika, ne specificira se i kako. Most između deklarativne prirode SQL jezika i potrebe definisanja načina pristupa podacima je rešen implementacijom *relacione algebre* [5]. LBDB sistem prevodi kod SQL programskog jezika u stablo operatora *relacione algebre*.

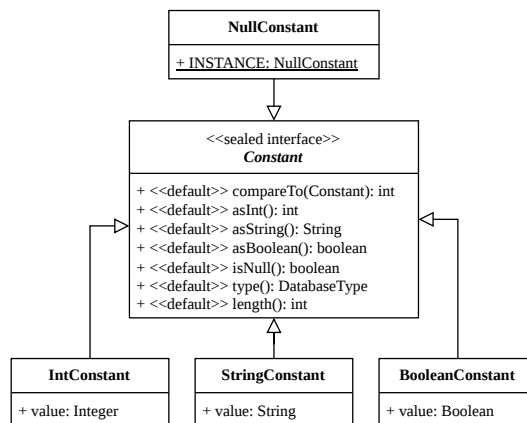
Relacija R je skup torki oblika $(d_1, d_2, \dots, d_j)$ gde za svaku komponentu d_k torke d_j važi $d_k \in D_k$, gde je D_k domen koji definiše skup svih dozvoljenih vrednosti za tu komponentu. U relacionim bazama podataka, tabela se modeluje kao relacija, dok operatori *relacione algebre* preslikavaju jednu ili više relacija u novu relaciju kao rezultat primenjene transformacije. Torke se mapiraju na slogove tabela.

5.1 Virtuelna mašina

Virtuelna mašina LBDB sistema predstavlja okruženje izvršavanja logičkih i aritmetičkih operacija i sastoji se od klasa koje modeluju komponente tih operacija.

5.1.1 Konstante

Sve vrednosti sa kojima relacioni operatori barataju predstavljene su Constant klasom. Ona omogućava sistemu da definiše generičko ponašanje za sve različite tipove podržane u sistemu. Relacioni operatori uvek vraćaju Constant objekte na svom izlazu. Implementira Comparable<Constant> interfejs zarad lakog poređenja.

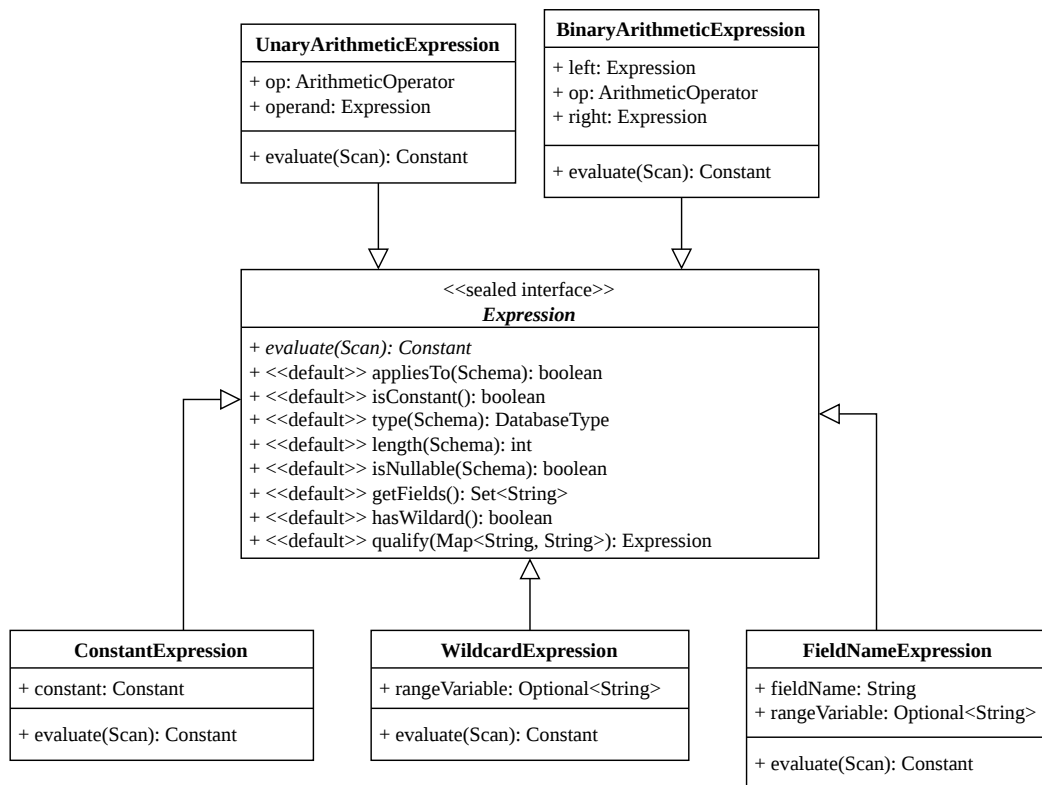


Slika 5: Hijerarhija konstanti

Interfejs konstante je definisan *sealed interface* Java konstruktom, zbog njegove odlične kompatibilnosti sa *switch* sintaksom. Konkretno konstante su predstavljene Java *record* strukturom. `NullConstant` nema nikakve podatke instance, jer su sve `NULL` vrednosti identične u sistemu, pa se svugde koristi ista instanca.

5.1.2 Izrazi

Sve aritmetičke operacije koje sistem evaluira su predstavljene `Expression` klasom. Evaluacija izraza uvek proizvodi `Constant` objekte. Izrazi se sastoje od proizvoljne kombinacije aritmetičkih operatora (+, -, *, /, ^), zagrada, konstanti i identifikatora kolona tabele. Interfejs izraza je definisan *sealed interface* Java konstruktom, zbog njegove odlične kompatibilnosti sa *switch* sintaksom. Konkretni izrazi su predstavljeni Java *record* strukturom.



Slika 6: Hijerarhija izraza

Evaluacija izraza predstavlja proces računanja konstante koja predstavlja vrednost tog izraza za neki slog tabele.

`BinaryArithmeticExpression` je izraz koji u sebi sadrži izraze i omogućava kreiranje stabla izraza, gde se prvo evaluiraju njegovi članovi, pa onda on. `UnaryArithmeticExpression` ima istu funkciju, ali za prefiksne operatore (+, -) i ima samo jedan član.

`WildcardExpression` suštinski ne predstavlja izraz, ali zbog načina [parsiranja izraza](#), tretira se kao jedan. Služi kao zamenski član (eng. *placeholder*) svih kolona upitanih tabela. Evaluacija je zabranjena operacija i baca izuzetak.

`FieldNameExpression` je izraz koji identifikuje virtuelnu kolonu neke tabele upita, sa opcionim preimenovanjem tabele i evaluira se na vrednost te kolone. `ConstantExpression` se evaluira direktno na konstantu za koju je vezan i nezavisan je od tabela.

Za izraze se definišu i pomoćne metode za proveru validnosti, kvalifikovanje i dobijanje opisnih atributa (tip, dužina, mogućnost sadržavanja *NULL* vrednosti).

5.1.3 Članovi

Član (eng. *term*) enkapsulira logiku za poređenje konstanti dva evaluirana izraza. Poređenje dve konstante zavisi od operatora poređenja koji mogu biti `=`, `!=`, `>`, `>=`, `<`, `<=`, `IS`, `IS NOT`. Razlika između `=` i `IS` (isto tako i između `!=` i `IS NOT`) je u tome kako se ponašaju sa *NULL* vrednostima. `IS` omogućava poređenje sa *NULL* vrednostima, dok `=` to ne podržava. Evaluacija člana nad nekim relacionim operatorom, za razliku od evaluacije izraza, vraća samo da li član važi ili ne.

5.1.4 Predikati

Predikati ulančavaju članove logičkim operatorima. Koriste se za uslov filtriranja. Evaluacija predikata nad nekim relacionim operatorom funkcioniše isto kao i evaluacija jednog člana, ali između tih članova stoje različite logičke operacije. LBDB sistem podržava samo `AND` logičke operatore između članova i ovo je jedna od glavnih ograničenja sistema.

TODO: limitaciju podržavanja samo `AND` u zaključku

5.2 Struktura relacionih operatora u sistemu

Za izvršavanje naredbi definisanih SQL jezikom, često je potrebno primeniti više relacionih operatora. Primena više relacionih operatora se radi njihovim ulančavanjem u stablovsku strukturu podataka i zbog ovoga se kaže da sistem izvršava “stablo” relacionih operatora.

5.2.1 Pajplajnovano procesovanje

Relacioni operatori podržani u sistemu imaju dve zajedničke osobine [\[1\]](#):

- generišu slogove jedan po jedan
- ne čuvaju generisane slogove i ne čuvaju nikakve međurezultate

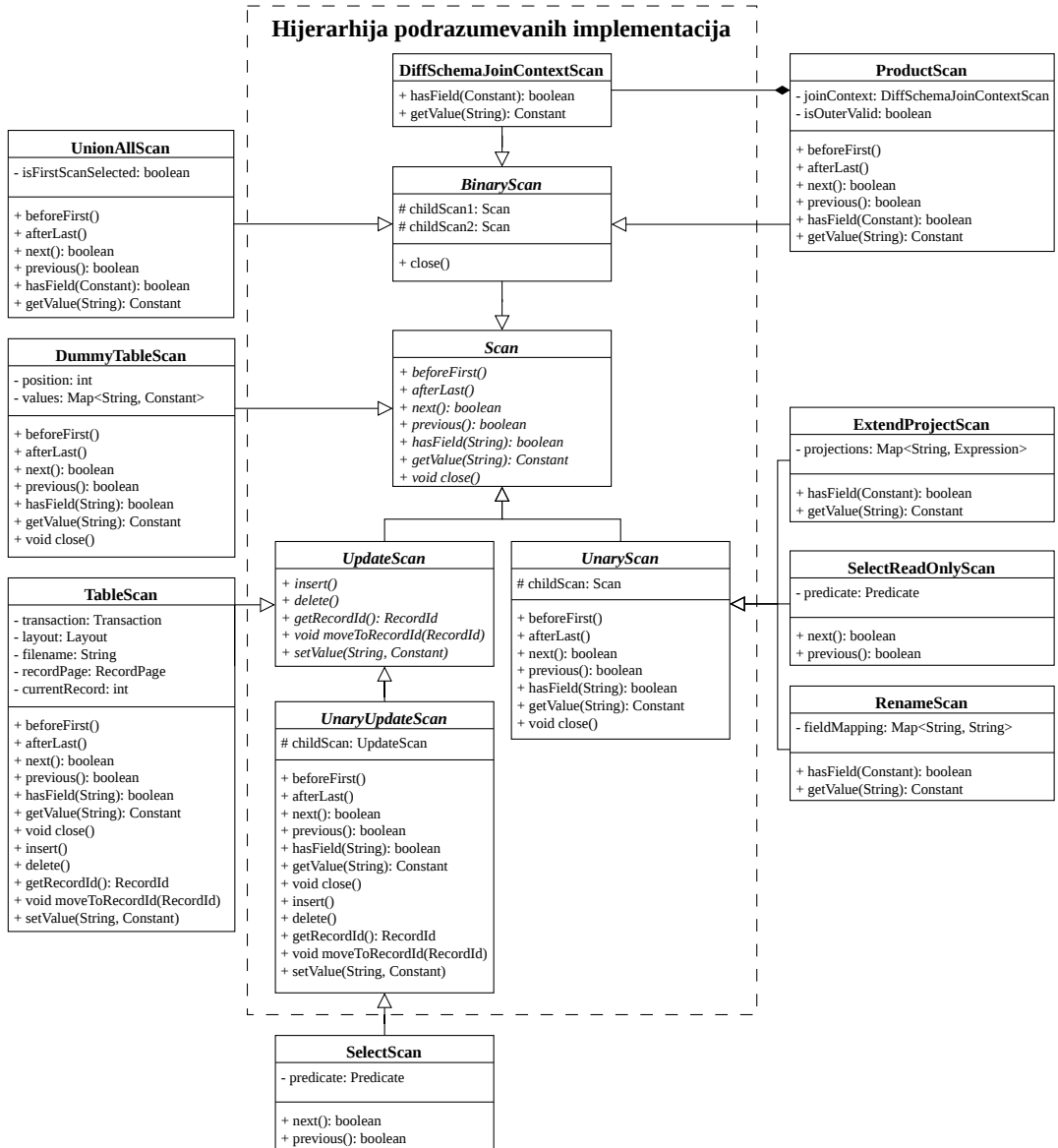
Zahtev za izvršenje neke operacije nad stablom operatora počinje od korena stabla, koji formira rezultat uz pomoć čvorova ispod njega, ali nekad i direktno. Ovim načinom, zahtev prolazi kroz celo stablo operatora. Vraća se greška klijentu ukoliko ni jedan čvor nije uspeo da formira rezultat zbog greške prilikom izvršavanja.

Kombinacija dve navedene osobine uz delegaciju operacija se zove pajplajnovano procesovanje (eng. *pipelined processing*). Korišćenje pajplajnovanog procesovanja u mnogim scenarijima ne dodaje nikakvo dodatno *U/I* opterećenje, pa ga je pogodno koristiti.

Prednost pajplajnovanog procesovanja što ne čuva međurezultate je upravo i njegova mana za određene operacije. Materijalizovano procesovanje (eng. *materialization*) omogućava i čuvanje međurezultata pa može rešiti ovu manu, ali dolazi sa svojim problemima. Takođe, neke operacije poput grupisane agregacije, vraćanje samo jedinstvenih slogova i proizvoljno sortiranje nije moguće obaviti bez materijalizovanog procesovanja. Potrebno je koristiti oba načina procesovanja u sistemu za najbolje rezultate, ali LBDB sistem implementira samo pajplajnovano procesovanje.

5.2.2 Hijerarhija implementacije relacionih operatora

Najopštija podela relacionih operatora je na one koji samo čitaju podatke (eng. *read-only*) i na one koji mogu da modifikuju podatke. Ova distinkcija je napravljena da se obezbedi sigurnost od pogrešne primene operatora u vremenu kompajliranja koda (eng. *compile time*). Hijerarhija podrazumevanih implementacija se sastoji od raznih kosturskih implementacija koje se koriste za lako definisanje novog relacionog operatora. Ostale klase van hijerarhije podrazumevanih implementacija predstavljaju različite relacione operatore podržane u sistemu i objašnjene su ispod.



Slika 7: Hijerarhija implementacije relacionih operatora

5.2.2.1 Scan

Scan apstraktna klasa definiše operacije neophodne za prolazak kroz sve vrednosti rezultujuće virtuelne tabele. Svaki relacioni operator implementira bar ove operacije. Vraćanje vrednosti se radi isključivo kroz Constant objekte. Scan se može, ali ne mora, mapirati na fizičku tabelu. Implementira AutoCloseable interfejs koji omogućava *RAII* (eng. *Resource Acquisition Is Initialization*) šablon, ali ne pruža sâmu logiku oslobađanja resursa.

5.2.2.2 UpdateScan

Rezultujuće virtualne tabele relacionih operatora koji dozvoljavaju modifikaciju vrednosti se moraju mapirati na fizičke tabele, jer nema smisla menjati virtualne vrednosti. To znači da za svaki virtualni slog r u stablu modifikujućih operatora, mora da postoji r' koji ima identičnu strukturu, poziciju i vrednosti u datoteci tabele. UpdateScan klasa definiše operacije promene vrednosti kolona nekog sloga, umetanja novog sloga i brisanja sloga.

5.2.2.3 UnaryScan

UnaryScan sadrži podrazumevane implementacije proizvoljnog relacionog operatora koji transformiše **jednu** relaciju, to jest jednu tabelu. Svaki poziv podrazumevane implementacije je samo prosleđen operatoru ispod. Ovakva struktura omogućava da operatori koji nasleđuju ovu klasu redefinišu samo metode koje su potrebne za funkcionisanje tog operatora i izbegava se dupliranje istog koda.

5.2.2.4 UnaryUpdateScan

UnaryUpdateScan sadrži podrazumevane implementacije proizvoljnog relacionog operatora koji može da modifikuje vrednosti fizičke tabele. Funkcioniše isto kao i UnaryScan i ima i sve podrazumevane implementacije iz njega.

5.2.2.5 BinaryScan

BinaryScan sadrži podrazumevane implementacije proizvoljnog relacionog operatora koji transformiše **dve** relacije, to jest dve tabele. Pošto operatori koji rade nad dve tabele nemaju toliko međusobnih preklapanja, BinaryScan implementira samo `close()` metodu koja oslobađa resurse oba deteta.

5.2.2.6 DiffSchemaJoinContextScan

DiffSchemaJoinContextScan ima sličnu ulogu kao BinaryScan, ali samo za operatore koji vrše multiplikativno objedinjavanje dve tabele i enkapsulira korektan pristup kolonama iz dve šeme koje nemaju preklapanje.

5.2.2.7 TableScan

TableScan nije pravi relacioni operator zato što njegova implementacija ne radi transformacije tabele, već pruža logiku za dobavljanje i modifikaciju fizičkih vrednosti. Služi kao omotač oko objekata stranice slogova i zadužena za konstruisanje novih povezanih objekata stranice slogova. Pošto pruža inicijalne vrednosti koje će dalje biti transformisane, uvek se nalazi na dnu stabla relacionih operatora.

5.2.2.8 DummyTableScan

DummyTableScan nije pravi relacioni operator zato što njegova implementacija ne radi transformacije tabele, već pruža logiku za dobavljanje i navigaciju jedinog sloga virtualne tabele koja se konstruiše u SQL upitima koji ne rade ni sa jednom fizičkom tabelom.

5.2.2.9 SelectScan

SelectScan implementira operator generalizovane selekcije $\sigma_{\varphi}(R)$ iz relacione algebre, gde je σ ime selekcionog operatora, φ je formula filtera, a R je relacija nad kojom se operator primenjuje. Redefiniše samo metode za prolazak kroz slogove, jer je to jedino neophodno da se ukinu slogovi koji ne prolaze filter. Sadrži Predicate objekat pomoću kog filtrira. Može da se koristi i u kontekstima modifikujućih stabala operatora i u kontekstima *read-only* stabala operatora i zbog toga postoji i SelectReadOnlyScan varijanta koja ima istu funkciju.

5.2.2.10 ExtendProjectScan

ExtendProjectScan implementira operator projekcije $\Pi_{a_1, \dots, a_n}(R)$ iz relacione algebre, gde je Π ime projekcionog operatora, a a_n predstavlja izraz čija evaluacija proizvodi vrednost komponente n . Operator projekcije koji klasa implementira nije striktno operator projekcije formalno definisan u relacionoj algebri, zato što dozvoljava kreiranje novih kolona sa izrazima koji će biti izražunati u trenutku izvršavanja stabla, a ne povučene direktno iz fizičke tabele. Dozvoljava i dodeljivanje proizvoljnog imena izrazima kolona, ali se to ne treba pomešati sa operatorom preimenovanja. Redefiniše metode dobavljanja vrednosti i provere postojanja kolone nekog imena.

5.2.2.11 RenameScan

RenameScan implementira operator preimenovanja $\rho_{a_n/b_n}(R)$ iz relacione algebre, gde je ρ ime operatora preimenovanja, a b_n predstavlja novo ime komponente a_n . Razlikuje se od formalne definicije operatora preimenovanja iz relacione algebre jer dozvoljava n preimenovanja od jednom da bi se izbeglo ulančavanje istih operatora. Bitno je napomenuti da operator preimenovanja omogućava pristup koloni sa imenom a kroz ime b , za razliku od operatora projekcije koji samo dodeljuje ime nekom izrazu. Redefiniše metode dobavljanja vrednosti i provere postojanja kolone nekog imena.

5.2.2.12 ProductScan

ProductScan implementira operaciju dekartovog proizvoda relacija R i S : $R \times S$. Rezultujuća relacija predstavlja virtuelnu tabelu koja ima $|R| * |S|$ torki, a svaka torka se sastoji od svih komponenti obe relacije. Operacija dekartovog proizvoda je multiplikativna, a ProductScan zahteva da tabele nemaju kolone sa istim imenom, pa se koristi DiffSchemaJoinContextScan. Iteracija kroz rezultujuću tabelu nakon ProductScan operatora se vrši tako što se za svaki slog prve tabele, prolazi kroz sve slogove druge tabele. Redefiniše metode dobavljanja vrednosti, provere postojanja kolone nekog imena i sve navigacione metode.

5.2.2.13 UnionAllScan

UnionAllScan implementira operaciju kreiranja relacije koje sadrži sve torke relacija R i S . Ne briše duplikate. Zahteva da su komponente torki obe relacije istog tipa i da obe

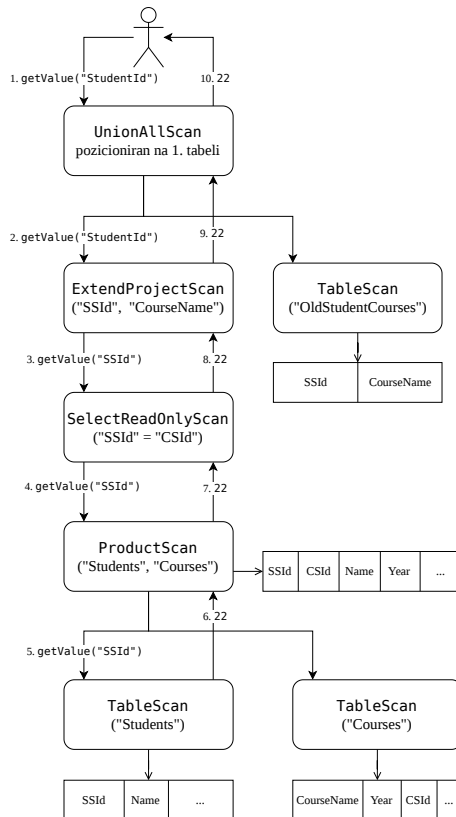
relacije imaju isti broj komponenata po torki. Po SQL standardu, kolonama druge tabele se pristupa po imenima prve. Operacija unije je aditivna. Iteracija kroz rezultujuću tabelu nakon UnionAllScan operatora se vrši tako što se prvo prolazi kroz sve slogove prve tabele, pa se prolazi kroz sve slogove druge tabele. Redefiniše metode dobavljanja vrednosti, provere postojanja kolone nekog imena i sve navigacione metode.

5.2.3 Primeri stabla relacionih operatora

Sledeći primeri pokazuju kako pozivi metoda putuju kroz stablo relacionih operatora. Primeri ne oslikavaju stabla relacionih operatora koje bi LBDB sistem napravio, već služe da pokažu pajplajnovano procesovanje i kako se Scan objekti oslanjaju jedni na druge.

5.2.3.1 Primer poziva getValue() metode

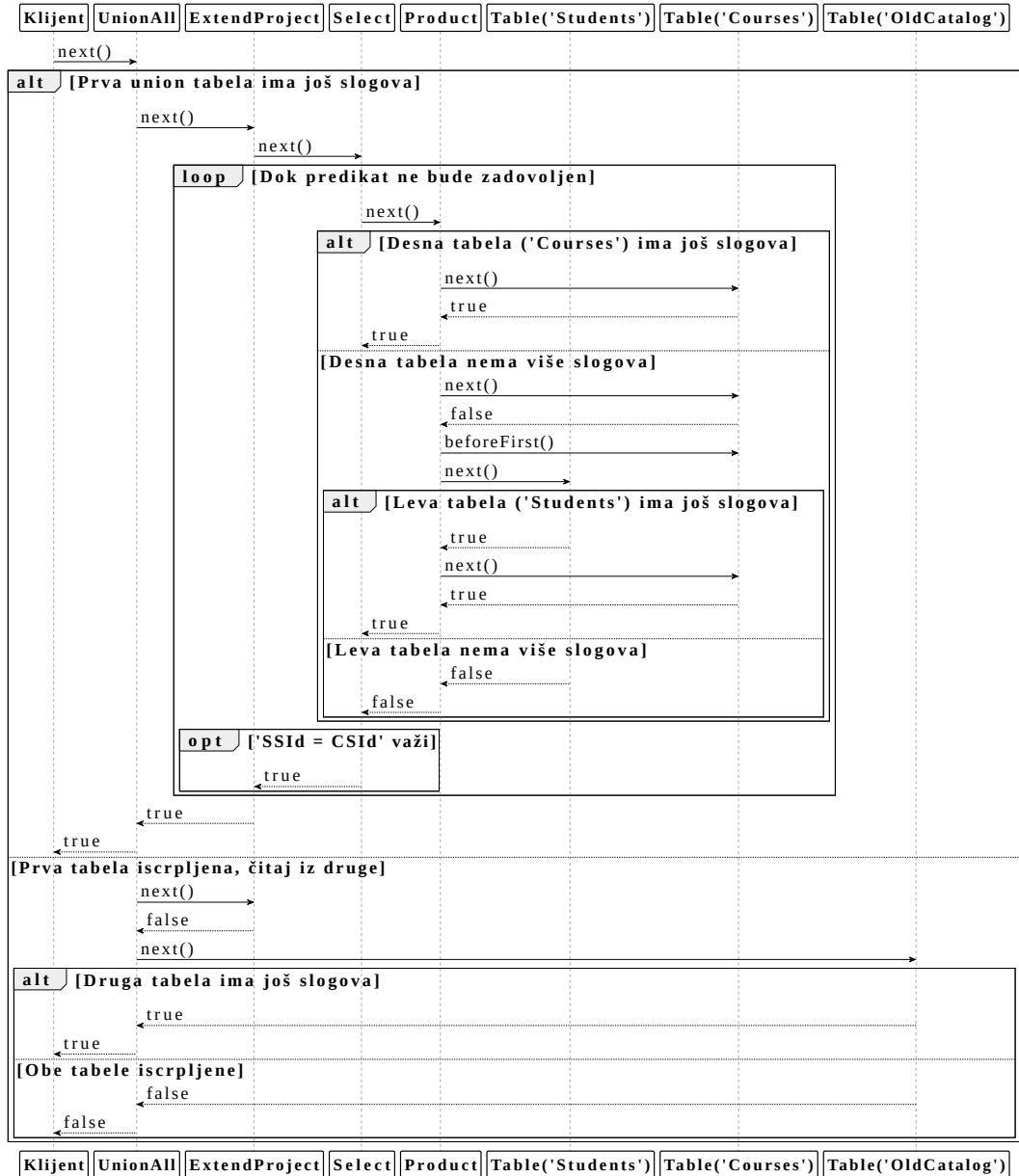
Prate se koraci poziva getValue() metode, za virtuelnu kolonu "StudentId". Union operator, koji je skroz na vrhu, spaja dve tabele: tabelu koja modeluje zastareli način vođenja evidencije i virtuelnu tabelu koja je rezultat podstabla operatora koji projektuju iste kolone kao i u tabeli zastarele evidencije. ExtendProjectScan je preimenovao "SSId" u "StudentId". Uz TableScan i ProductScan operatore stoji i uprošćena šema.



Slika 8: Primer poziva getValue() metode u konkretnom stablu relacionih operatora

5.2.3.2 Primer poziva next() metode

Ovaj primer koristi isto relaciono stablo kao i prethodni primer. Pomoću dijagrama sekvence se opisuje procedura iteracije kroz stablo sa svim vrstama operatora koji menjaju podrazumevanu iteraciju.



Slika 9: Dijagram sekvence next() metode u konkretnom stablu relacionih operatora

Glava 6

Parsiranje

Klijenti šalju SQL naredbe u tekstualnom formatu, ali sistemu komad teksta nema nikakvo inherentno značenje. Podsystem za ekstrakciju informacija iz teksta naredbe se zove parser.

Nije svaki komad teksta validna SQL naredba, ali njegova validnost se može podeliti na dva sloja [1]:

- sintaktička validnost, gde sintaksa predstavlja skup pravila koja definišu moguće operacije po nekoj gramatici,
- semantička validnost, koja je ispunjena ako je neka operacija validna u kontekstu podataka koje koristi (imena tabela, imena kolona, ...)

Parsiranje konstruiše apstraktno sintaktičko stablo (eng. *abstract syntax tree*, *AST*) koje se mapira na podržane operacije i služi za proveru **samo** sintaktičke validnosti operacije. Provera semantičke validnosti je [deo planera](#).

6.1 Tokenizator

Blokovi teksta se sastoje od individualnih karaktera. Većina individualnih karaktera sadrži jako malo značenja kada se obrađuju nezavisno i zbog toga se uvodi sistem koji grupiše povezane karaktere. Skup grupisanih karaktera koji zajedno imaju visok stepen značenja se zovu tokeni. Karakteri koji se obrađuju sami su isto opisani kao tokeni, jer je bitno da je svaki token imenovan.

Tokenizer klasa sadrži logiku pretvaranja bloka teksta u tokene odgovarajućeg tipa i implementirana je preko Java Iterator interfejsa. Token je definisan *sealed interface* Java konstruktom, zbog njegove odlične kompatibilnosti sa switch sintaksom. Svi tokeni u sistemu su grupisani u sledeće kategorije:

- ključne reči SQL jezika,
- identifikatori,
- simboli,
- brojevi,
- *String*-ovi,
- nevalidni karakteri,
- EOF token.

Svaka kategorija tokena implementira Token interfejs i predstavljena je *record* ili *enum* Java strukturom.

6.2 Parser

Gramatika nekog jezika predstavlja skup pravila koje opisuju sve legalne komade teksta, koje neki sistem podržava. Sintaktičke kategorije su koncepti sa kojima gramatika barata. Predstavljaju čvorove sintaktičkog stabla i sadrže se od drugih sintaktičkih kategorija i tokena.

Vrsta parsiranja koja je implementirana se zove *recursive descent* parsiranje. U *recursive descent* parsiranju, gramatika se proverava od gore ka dole. Ulazna tačka je koren sintaktičkog stabla i za svako podstablo, to jest gramatičko pravilo, postoji funkcija obrađuje to pravilo. Funkcije se često pozivaju rekurzivno da bi obradili veće celine, otud i ime ovog načina parsiranja. Svaka funkcija obrade pravila, na bilo kom nivou, se mapira na jednu sintaktičku kategoriju.

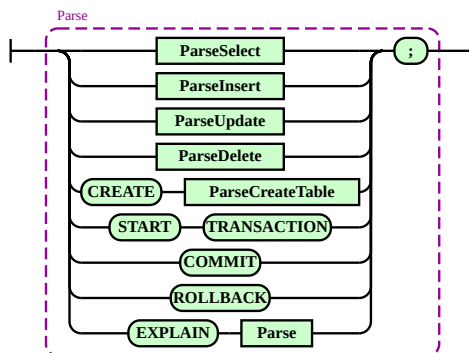
6.2.1 Iskazi

Uspešno parsiranje nekog bloka teksta koji predstavlja SQL operaciju rezultuje u iskazu, koji sadrži sve neophodne podatke da se ta operacija izvrši. Iskaz (Statement klasa) je definisan *sealed interface* Java konstruktom, zbog njegove odlične kompatibilnosti sa *switch* sintaksom. Svaki iskaz je predstavljen Java *record* strukturom i nasleđuje *Statement*.

6.2.2 Sintaktičke kategorije SQL operacija

Funkcije koje generišu iskaze predstavljaju sintaktičke kategorije najvišeg apstrakcionog nivoa i grupisane su u različite Java datoteke. Svaka datoteka sadrži sve sintaktičke kategorije nižeg apstrakcionog nivoa potrebne da se iskaz uspešno obradi.

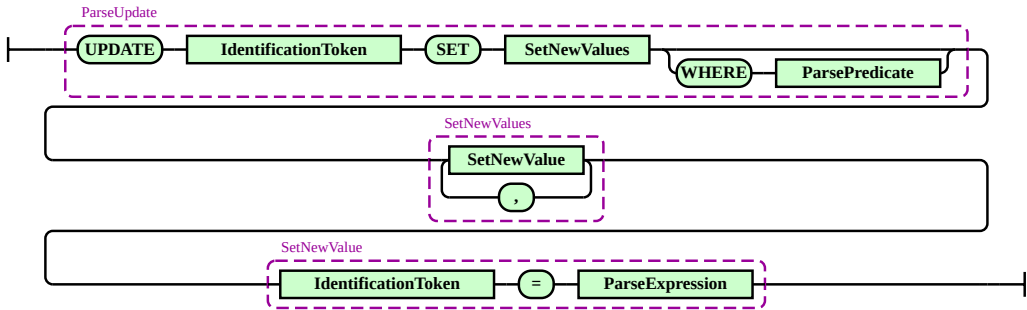
6.2.2.1 Parse



Slika 10: Gramatika Parse sintaktičke kategorije

Parse predstavlja glavnu sintatičku kategoriju i grupiše sve ostale sintaktičke kategorije. Omogućava i [EXPLAIN](#) naredbu, koja generiše opis naredbe koja će se izvršiti. Iskazi upravljanja životnim ciklusima transakcija se isto parsiraju ovde jer su previše jednostavni da bi se pravila posebna sintaktička kategorija. Vraća *Statement* objekat.

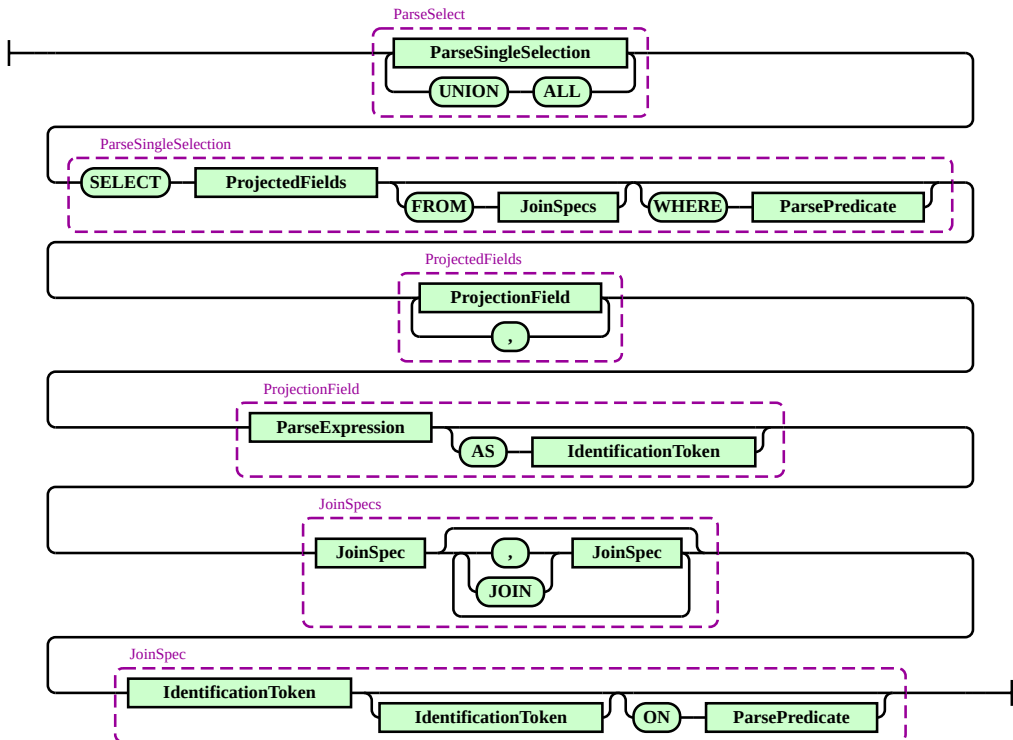
6.2.2.2 ParseUpdate



Slika 11: Gramatika ParseUpdate sintaktičke kategorije

ParseUpdate predstavlja naredbu modifikovanja podataka neke tabele. Može sadržati proizvoljan broj novih dodela vrednosti, ali svako polje u svim dodelama vrednosti mora postojati u referenciranoj tabeli. Moguće je modifikovati samo neke slogove, a ne sve, tako što se definiše uslov pretrage. Vraća UpdateStatement objekat.

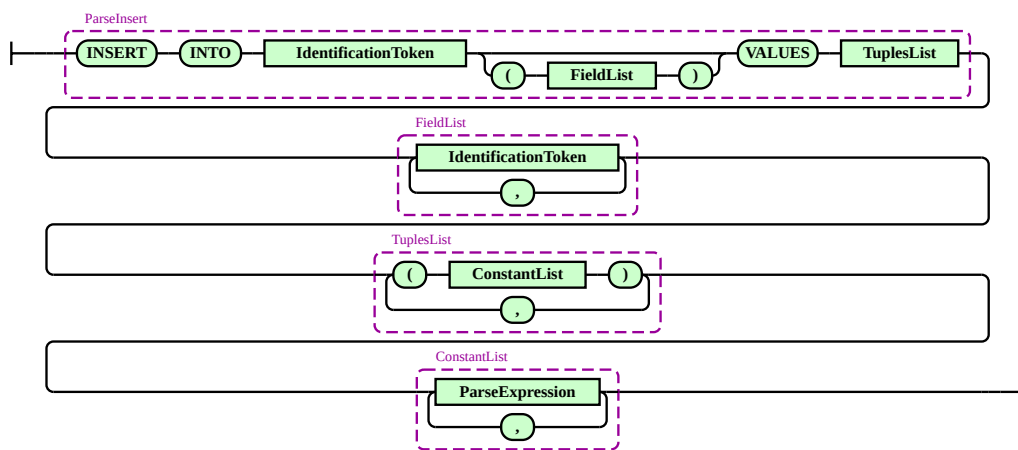
6.2.2.3 ParseSelect



Slika 12: Gramatika ParseSelect sintaktičke kategorije

ParseSelect predstavlja naredbu upita (eng. *query*) podataka. Može sadržati proizvoljan broj projektovanih kolona, gde je svaka kolona predstavljena izrazom kome se može dodeliti neko ime (uz AS ključnu reč). Podržava filtriranje na osnovu uslova pretrage. Upit može biti nad pravim tabelama ili nad [virtuelnom tabelom koja sadrži jedan slog](#). Ulančavanje tabela se može raditi na dva načina: samo navođenje tabela odvojene zarezom ili preko JOIN ključne reči gde se uslov ulančavanja upisuje odmah. Uslov ulančavanja napisan u JOIN sekciji se samo dodaje na uslov pretrage, umesto da predstavlja neki specijalan način ulančavanja. **TODO: Jedno od glavnih ograničenja sistema je da projektovane kolone ne mogu da sadrže logička računanja**, već samo aritmetička. Vraća SelectStatement objekat.

6.2.2.4 ParseInsert



Slika 13: Gramatika ParseInsert sintaktičke kategorije

ParseInsert predstavlja naredbu umetanja novih slogova u neku tabelu. Lista polja ne mora biti definisana, uzima se podrazumevani redosled koji je napravljen tokom kreiranja te tabele. Izrazi moraju biti konstantni, to jest njihova evaluacija ne sme zavisiti od vrednosti koje ne mogu da se izračunaju bez pristupa tabelama. Vraća InsertStatement objekat.

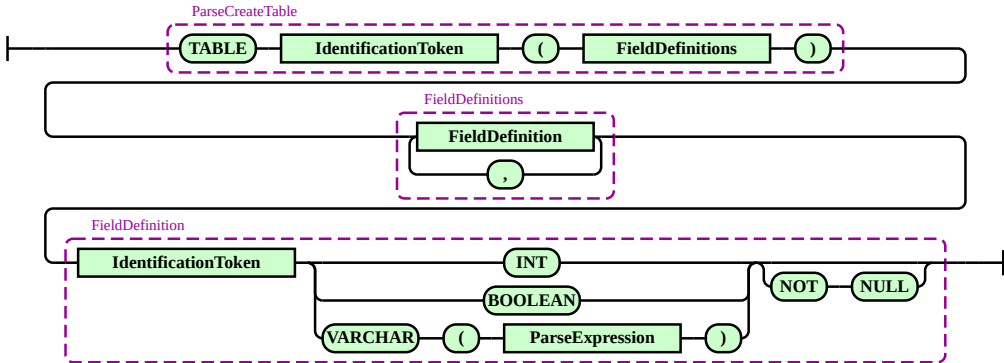
6.2.2.5 ParseDelete



Slika 14: Gramatika ParseDelete sintaktičke kategorije

ParseDelete predstavlja naredbu brisanja slogova iz neke tabele. Moguće je obrisati samo slogove koji ispunjavaju neki uslov, tako što se definiše uslov pretrage. Vraća DeleteStatement objekat.

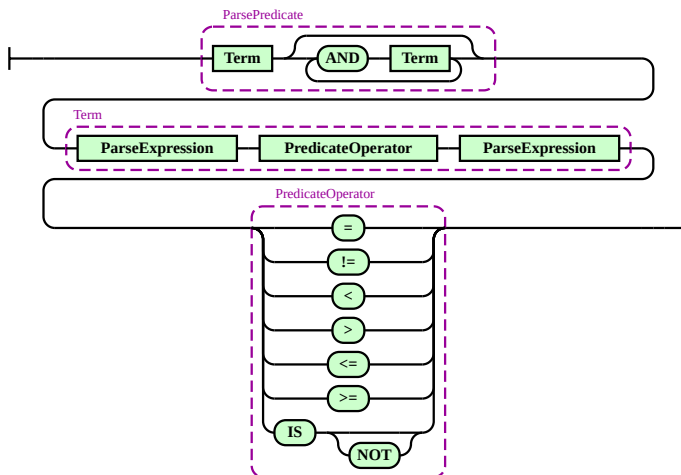
6.2.2.6 ParseCreateTable



Slika 15: Gramatika ParseCreateTable sintaktičke kategorije

ParseCreateTable predstavlja naredbu kreiranja nove tabele. Tabela može sadržati maksimalno **31 polje**. Polje može biti jedno od tipova **podržanih u sistemu**, a za *String* (VARCHAR) tip se mora definisati i maksimalna dužina, koja mora biti konstantan izraz. Ograničenje da slogovi za neku kolonu ne smeju imati *NULL* vrednosti je opciono i definiše se nakon tipa kolone. Vraća CreateTableStatement objekat.

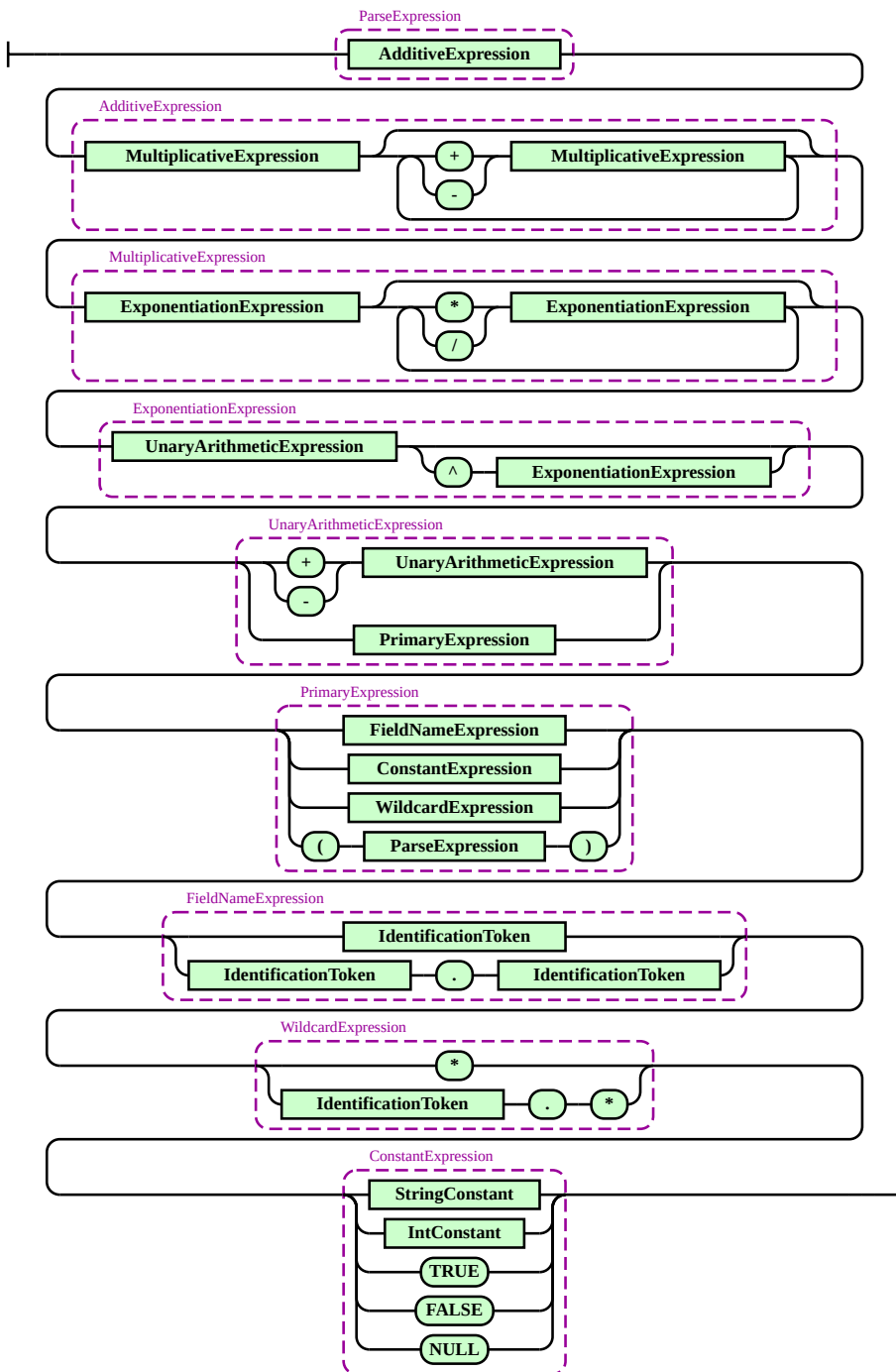
6.2.2.7 ParsePredicate



Slika 16: Gramatika ParsePredicate sintaktičke kategorije

ParsePredicate je specijalna vrsta sintaktičke kategorije koja ne proizvodi iskaz, već služi za kreiranje sintaktičkih stabala **predikata**. Parsiraju se izrazi i operacije poređenja od kojih se članovi sastoje, a zatim se ulančavaju članovi da bi se formirao predikat. Vraća Predicate objekat.

6.2.2.8 ParseExpression

Slika 17: Gramatika `ParseExpression` sintaktičke kategorije

`ParseExpression` je specijalna vrsta sintaktičke kategorije koja ne proizvodi iskaz, već služi za kreiranje sintaktičkih stabala [izraza](#). Parsiranje izraza je urađeno specijalnom tehnikom *recursive descent* parsiranja koja se zove *Pratt parsing* [6]. *Pratt parsing* definiše tehnike obrade prioriteta operacija, zagrada, prepoznavanja identifikatora i zamenskih članova. Vraća `Expression` objekat.

Prvo se parsira prefiksni izraz, koji može biti literal različitog tipa, identifikator, zamenski član, izraz sa unarnom operacijom ili izraz u zagradama. Rezultat ovog koraka postaje početni levi operand. Zatim se ulazi u petlju koja se izvršava sve dok je prioritet sledećeg (infiksnog) operatora strogo veći od trenutnog prioriteta.

Unutar petlje, operator se konzumira, a desni operand se dobija rekurzivnim pozivom funkcije za parsiranje izraza kojoj se prosleđuje prioritet tog novog operatora (ili prioritet umanjen za jedan, ukoliko je operacija desno-asocijativna, poput stepenovanja). Od levog operanda, operatora i desnog operanda kreira se novo stablo binarnog izraza, koje zatim postaje novi levi operand za narednu iteraciju petlje.

U isečku koda ispod se mogu videti različiti prioriteti operacija na osnovu tokena koji ih opisuju. Token `STAR` predstavlja i operator zamenskog člana, pa se zove `STAR` umesto `MULTIPLY`. Prefiksne operacije imaju najveći prioritet.

```
private static final int PREFIX_PRECEDENCE = 100;

private int getPrecedence(Token opToken) {
    return switch (opToken) {
        case SymbolToken.CARET -> 30;
        case SymbolToken.STAR, SymbolToken.DIVIDE -> 20;
        case SymbolToken.PLUS, SymbolToken.MINUS -> 10;
        default -> 0;
    };
}
```

Listing 7: Prioriteti aritmetičkih operacija u sistemu

Glava 7

Planiranje

Planer i planovi čine podsistem planiranja koji je jedan od tri glavna podsistema LBDB sistema [obrade upita](#). U okviru životnog ciklusa obrade jedne SQL naredbe, podsistem planiranja se nalazi između podsistema za parsiranje i stabla relacionih operatora.

7.1 Struktura planova u sistemu

Za svaki relacioni operator se definiše njegov plan, a za svako stablo relacionih operatora se definiše stablo planova. Plan relacionog operatora opisuje šemu nakon primene operatora i omogućava računanje [statističkih metapodataka](#) za rezultujuću virtuelnu tabelu. Ovi statistički metapodaci su jedan deo podataka koje algoritmi za konstrukciju stabla planova koriste za efikasan rad.

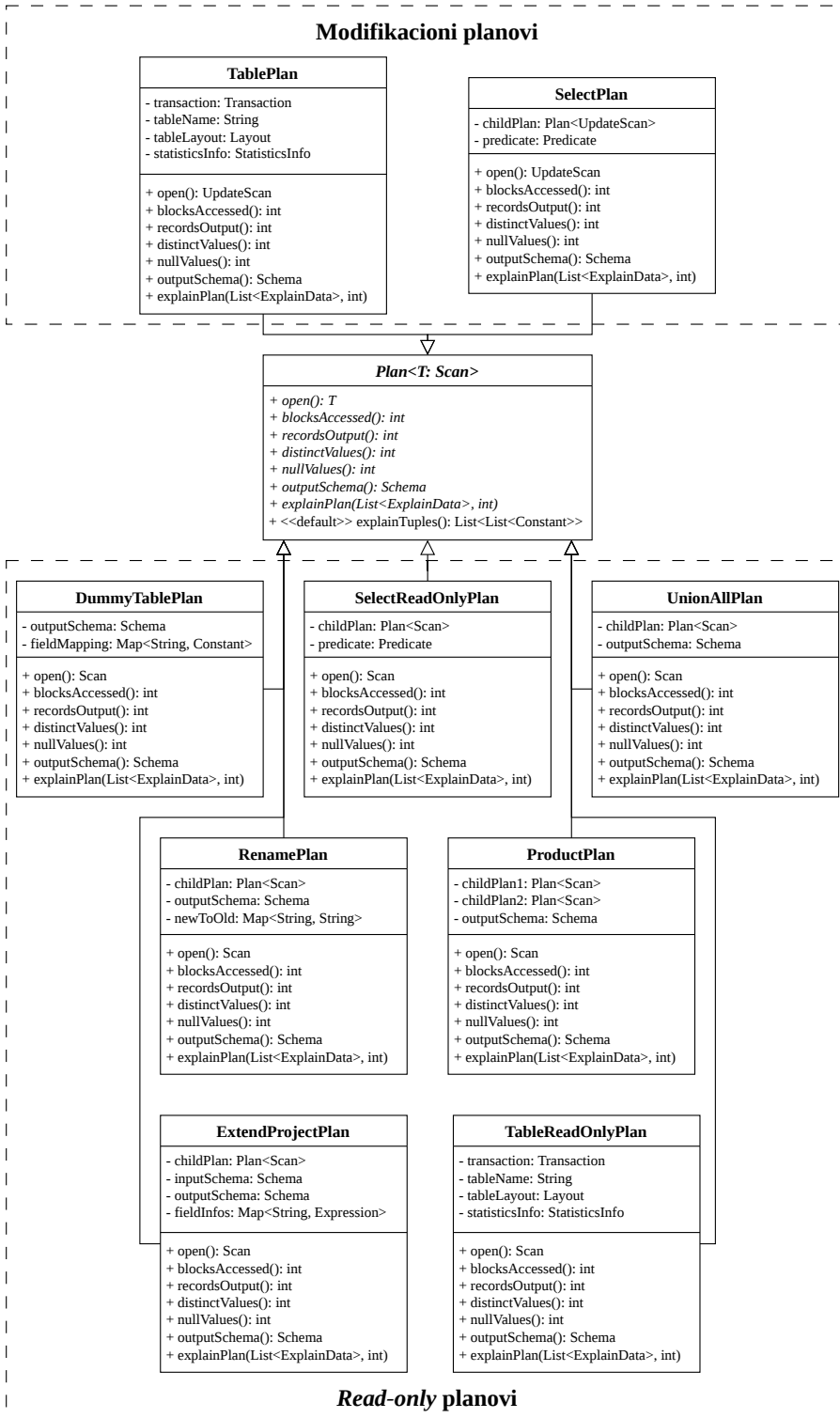
Statističke metapodatke koje plan može da izračuna su isti kao i statistički metapodaci koji se prate za fizičke tabele, a oni su:

- broj blokova potrebnih za prolazak kroz sve slogove,
- broj slogova,
- broj jedinstvenih vrednosti za svaku kolonu,
- broj *NULL* vrednosti za svaku kolonu.

Izračunati statistički podaci [nisu 100% precizni](#), ali bez obzira na to, pomažu algoritmima konstrukcije planova (planerima).

7.1.1 Hijerarhija implementacije planova

Najopštija podela planova je na one koji samo čitaju podatke (eng. *read-only*) i na one koji mogu da modifikuju podatke, po [hijerarhiji relacionih operatora](#). Za razliku od hijerarhije relacionih operatora, ne postoji hijerarhija podrazumevanih implementacija jer klase planova nemaju toliko zajedničkih osobina. Podela na *read-only* i modifikacione planove je odrađena preko *generics* Java konstrukta, umesto deljenja glavnog interfejsa na dva podtipa.



Slika 18: Hijerarhija implementacije planova

7.1.1.1 Plan

Plan interfejs definiše operacije neophone za računanje svih statističkih podataka, dobijanje šeme rezultujuće tabele, pretvaranje stabla planova u stablo relacionih operatora i dobijanje slogova za [automatski opis plana](#).

7.1.1.2 TablePlan

TablePlan opisuje konkretnu fizičku tabelu, umesto da vrši transformacije virtuelne tabele. Izlazna šema je jednaka šemi fizičke tabele. Izvlači statističke podatke direktno iz menadžera metapodataka za tabelu za koju je vezan. Statistički podaci [nisu ažurni](#), ali pružaju dovoljno dobru statistiku za potrebe LBDB sistema. Izračunati statistički metapodaci svih planova u stablu planova eventualno zavise od vrednosti statističkih metapodataka ovog plana. Može da se koristi i u kontekstima modifikujućih stabala operatora i u kontekstima *read-only* stabala operatora i zbog toga postoji i TableReadOnlyPlan varijanta koja ima istu funkciju.

7.1.1.3 DummyTablePlan

DummyTablePlan opisuje virtuelnu tabelu koja se sastoji od jednog sloga u upitima koji ne rade sa fizičkim tabelama. Izlazna šema se određuje na osnovu konstantnih vrednosti u upitu, a statistički podaci su precizni jer se lako računaju pošto je broj konkretnih vrednosti jako mali.

7.1.1.4 SelectPlan

SelectPlan opisuje virtuelnu tabelu nakon primene uslova filtriranja. Izlazna šema je jednaka šemi podređenog plana. Broj blokova ostaje nepromenjen jer da bi znali koji sve slogovi ispunjavaju uslov filtera, potrebno je proći kroz sve slogove, pa sa time i kroz sve blokove podređenog plana.

Redukcioni faktor predstavlja za koliko puta će broj slogova na izlazu biti smanjen. Računa se na osnovu uslova filtriranja, koji je predstavljen [predikatom](#). Svaki član predikata predstavlja jedan deo filtera i ima svoj redukcioni faktor. Pošto sistem podržava samo ulančavanje članova AND logičkim operatorom, redukcioni faktor predikata se računa kao proizvod svih redukcionih faktora članova od kog se predikat sastoji.

Algoritam računanja redukcionog faktora jednog člana je predstavljen na [dijagramu](#). Pošto se broj slogova nakon filtriranja dobija deljenjem broja slogova podređenog plana i redukcionog faktora, specijalni slučajevi se mogu predstaviti različitim konstantama.

Specijalni slučaj nejednakosti je predstavljen konstantom *NEJEDNAKOSTI* koja ima vrednost 3.0 i označava procenjenu vrednost redukcije u slučaju korišćenja operacija nejednakosti.

Specijalni slučaj kada postoje prekompleksni izrazi je predstavljen konstantom *KOMPLEKSNO* koja ima vrednost 10.0 i označava procenjenu vrednost redukcije u slučaju

Procena broja jedinstvenih vrednosti za izlaznu kolonu vrši se analizom predikata i pronalaženjem uslova jednakosti i on je jednak:

- 0, ukoliko predikat izjednačava traženu kolonu sa dve ili više različitih konstanti. U ovom slučaju, uslov je kontradiktoran i ni jedan slog neće zadovoljiti filter, pa samim tim neće biti ni jedinstvenih vrednosti,
- 1, ukoliko predikat izjednačava traženu kolonu sa tačno jednom konstantom. Svi slogovi koji prođu filter imaju istu vrednost za tu kolonu,
- minimumu između broja jedinstvenih vrednosti te kolone i svih kolona sa kojima je izjednačena, ukoliko kolona nije izjednačena ni sa jednom konstantom, ali jeste sa jednom ili više drugih kolona. U ovom slučaju, broj jedinstvenih vrednosti tražene kolone ne može biti veći od njenog originalnog broja jedinstvenih vrednosti iz podređenog plana, ali ne može biti veći ni od broja jedinstvenih vrednosti najrestriktivnije kolone sa kojom je izjednačena.

Procena broja *NULL* vrednosti za svaku izlaznu kolonu vrši se analizom predikata i njegovog odnosa prema *NULL* konstantama i on je jednak:

- ukupnom broju izlaznih slogova ovog plana, ukoliko predikat eksplicitno izjednačava traženu kolonu sa *NULL* vrednošću, a izlazna šema dozvoljava *NULL* vrednosti za tu kolonu. U ovom slučaju, svi slogovi koji prođu filter imaju *NULL* vrednost,
- 0, ukoliko predikat izjednačava traženu kolonu sa *NULL* vrednošću, ali izlazna šema ne dozvoljava *NULL* vrednosti (nije *nullable*). U ovom slučaju, uslov je nemoguće ispuniti,
- 0, ukoliko predikat sadrži uslov koji eksplicitno isključuje *NULL* vrednosti za traženu kolonu. Svi slogovi sa *NULL* vrednostima će pasti na ovakvom filteru,
- proporciji broju *NULL* vrednosti iz podređenog plana, ukoliko predikat ne spominje *NULL* konstantu u vezi sa traženom kolonom. U ovom slučaju, pretpostavlja se da uslov filtera ravnomerno smanjuje ukupan broj slogova i broj *NULL* vrednosti, pa se broj *NULL* vrednosti iz podređenog plana deli sa faktorom redukcije celokupnog predikata.

Može da se koristi i u kontekstima modifikujućih stabala operatora i u kontekstima *read-only* stabala operatora i zbog toga postoji i *SelectReadOnlyPlan* varijanta koja ima istu funkciju.

7.1.1.5 ExtendProjectPlan

ExtendProjectPlan opisuje virtuelnu tabelu sa svim projektovanim kolonama. Izlazna šema ima sve dodate kolone, a iz nje su izbrisane neprojektovane kolone. S obzirom da operacija projekcije ne dodaje nove slogove, broj blokova i broj slogova ostaju nepromenjeni i direktno se preuzimaju od podređenog plana.

Procena broja jedinstvenih vrednosti za svaku izlaznu kolonu vrši se na osnovu složenosti izraza koji tu kolonu definiše i on je jednak:

- 0, ukoliko se traži procena za kolonu koja se ne nalazi u projekciji,
- 1, ukoliko je izraz konstanta (ne referencira ni jednu kolonu),

- broju jedinstvenih vrednosti te kolone iz podređenog plana, ukoliko izraz referencira tačno jednu kolonu. Pretpostavka je da većina transformacija nad jednom kolonom (npr. aritmetičke operacije) zadržava sličnu distribuciju vrednosti,
- ukupnom broju slogova, ukoliko izraz referencira više od jedne kolone. U ovom slučaju, pretpostavlja se da kombinacija više polja rezultuje jedinstvenom vrednošću za svaki slog.

Procena broja *NULL* vrednosti za svaku izlaznu kolonu vrši se na osnovu složenosti izraza koji tu kolonu definiše i on je jednak:

- 0, ukoliko se traži procena za kolonu koja se ne nalazi u projekciji,
- 0, ukoliko šema garantuje da izraz ne može imati *NULL* vrednosti (nije *nullable*),
- 0 ukoliko je izraz jednak bilo kojoj konstanti sem *NULL* konstante,
- 1 ukoliko je izraz jednak *NULL* konstanti,
- broju jedinstvenih vrednosti te kolone iz podređenog plana, ukoliko izraz referencira tačno jednu kolonu. Pretpostavka je da većina transformacija nad jednom kolonom zadržava sličnu distribuciju *NULL* vrednosti,
- maksimalanom broju *NULL* vrednosti među svim referenciranim kolonama iz podređenog plana, ukoliko izraz referencira više od jedne kolone. Pretpostavlja se da će izraz biti *NULL* ukoliko je barem jedan od operanada *NULL*, pa je maksimalan broj *NULL* vrednosti pesimistična procena.

7.1.1.6 RenamePlan

RenamePlan opisuje virtuelnu tabelu sa svim primenjenim preimenovanjima kolona. Izlazna šema sadrži sve kolone sa novim imenom i ni jednu kolonu sa starim imenom. S obzirom da operacija preimenovanja ne dodaje nove slogove, broj blokova i broj slogova ostaju nepromenjeni i direktno se preuzimaju od podređenog plana. Procena broja jedinstvenih vrednosti kolone je jednaka podređenom planu ukoliko se traži novo ime stare kolone ili ukoliko je kolona nepreimenovana, a jednaka je 0 ako se traži staro ime preimenovane kolone. Procena broja *NULL* vrednosti funkcioniše isto.

7.1.1.7 ProductPlan

ProductPlan opisuje virtuelnu tabelu koja je proizvod dve tabele. Izlazna šema sadrži sve kolone od obe tabele.

Za demonstraciju računice broja blokova kod proizvoda dve tabele, definišemo T_1 i T_2 :

Tabela 4: Primer karakteristika tabela za računanje broja blokova plana proizvoda

T_i	$B(T_i)$	$R(T_i)$	$RPB(T_i)$
T_1	5	1000	$\frac{1000}{5} = 200$
T_2	100	500	$\frac{500}{100} = 5$

- $B(T_i)$ predstavlja broj blokova neke tabele,
- $R(T_i)$ predstavlja broj slogova neke tabele,
- $RPB(T_i)$ predstavlja koliko slogova može da stane po jednom bloku za neku tabelu.

Ovaj primer se odnosi na rad sa konkretnim fizičkim tabelama, ali u generalnom slučaju, tabele nisu fizičke, već su predstavljene podređenim planovima sa kojima plan proizvoda barata.

Da bi se prošlo kroz svaki slog rezultujuće tabele, potrebno je da za se svaki slog leve tabele prođe kroz svaki slog desne tabele. Formula koja opisuje broj blokova potreban da se ovo izvrši je sledeća [1]:

$$B(T_r) = B(T_l) + (R(T_l) * B(T_d))$$

Ako stavimo konkretne vrednosti tabela T_1 i T_2 u ovu formulu, dobijamo različite rezultate u odnosu na to koja tabela je leva, a koja desna:

- $(T_l = T_1, T_d = T_2) \Rightarrow B(T_r) = 5 + (1000 * 100) = 100005$
- $(T_l = T_2, T_d = T_1) \Rightarrow B(T_r) = 100 + (500 * 5) = 2600$

Vidi se da ako stavimo da tabela T_1 bude desna, a T_2 leva, dobijamo manji broj blokova rezultujuće tabele, a sa time i efikasniju operaciju proizvoda. Ekvivalentna formula [1]:

$$B(T_r) = B(T_l) + (RPB(T_l) * B(T_l) * B(T_d))$$

daje bolji uvid zbog čega računica broja blokova rezultujuće tabele nije simetrična u odnosu na dve tabele koje učestvuju u proizvodu. Sabirak $RPB(T_l) * B(T_l) * B(T_d)$ znatno više utiče na finalni rezultat u odnosu na $B(T_l)$.

Što je slog manji, jedan blok može da ih sadrži više. U suprotnom, što je slog veći, jedan blok može da ih sadrži manje. U tabelama gde je slog veći, potrebno je pristupiti više blokova da bi se prošlo kroz isti broj slogova kao u tabelama gde je slog manji. U operacijama proizvoda bolje je staviti tabelu gde je slog veći (to jest gde je RPB manji) na levu stranu, a tabelu gde je slog manji (to jest gde je RPB veći) na desnu stranu jer se slogovima desne tabele pristupa znatno više nego slogovima leve tabele.

Broj slogova je proizvod broja slogova oba podređena plana, a broj jedinstvenih i broj *NULL* vrednosti se prosleđuje podređenom planu u kom se nalazi tražena kolona.

7.1.1.8 UnionAllPlan

UnionAllPlan opisuje virtuelnu tabelu koja je zbir dve tabele. Izlazna šema je jednaka izlaznoj šemi levog podređenog plana. Broj blokova je zbir broja blokova oba podređena plana, a broj slogova je zbir broja slogova oba podređena plana. Procena broja jedinstvenih vrednosti kolone je jednaka zbiru procena jedinstvenih vrednosti oba podređena plana za

tu kolonu. Procena *NULL* vrednosti kolone je jednaka zbiru procena *NULL* vrednosti oba podređena plana.

7.2 Planer

Većina naredni definisanih SQL standardom zahteva propratno stablo relacionih operatora. Konstrukcija i analiza stabala je posao planera, ali pored toga planer vrši i proveru semantičke tačnosti svih naredbi.

Glavna podela tehnika planiranja u relacionim bazama podataka je na tehnike praćenja striktnih pravila pravljenja planova (eng. *rule-based optimisation*, *RBO*; *heuristics-based optimisation*, *HBO*) i tehnike planiranja koji rade sa cenama (eng. *cost-based optimisation*, *CBO*). Cena predstavlja kombinaciju statističkih metapodataka relacionih operatora sa hardverskim osobinama koji ti relacioni operatori koriste.

Raniji sistemi upravljanja bazama podataka poput *INGRES* sistema su koristili *RBO* tehnike planiranja [8], dok moderni sistemi koriste *CBO* tehnike planiranja^{2,3} koji su postali popularni nakon *System R* istraživačkog papira [7].

Evolucija tehnika planiranja, koja se može videti kroz ovu glavnu podelu, postoji jer je kroz istoriju bilo potrebno obezbediti sve efikasnije planere koji rade sa sve većim skupovima podataka.

7.2.1 Evaluacija izraza tokom planiranja

Evaluacija izraza u stablu relacionih operatora je najskuplje mesto evaluacije, jer se operacije izvršavaju u okviru virtuelne mašine sistema, gde se ne koriste procesorske instrukcije direktno. *PartialEvaluator* omogućava da se izrazi sa trivijalnim operacijama redukuju na čistu konstantu čija evaluacija ne zahteva obradu na virtuelnoj mašini.

Trivijalna operacija se definiše kao aritmetička operacija koja ne transformiše podatke ili kao aritmetička operacija između dve konstante.

Predikati se sastoje od članova, koji se sastoje od izraza, tako da *PartialEvaluator* može da redukuje i njih. Nije podržana redukcija trivijalnih logičkih operacija.

7.2.2 Ulazna tačka kreiranja i izvršavanja planova

Svaka SQL naredba, koja je prvobitno niz karaktera, se prosleđuje *Planner* klasi, koja je dalje obrađuje. Klasa *Planner* definiše dve grupe funkcija koje su prilagođene različitim *API* interfejsima. Obe grupe funkcija znaju da barataju sa podsistemom parsiranja, koji pretvara niz karaktera u *Statement* objekat. Grupe se sastoje od funkcija:

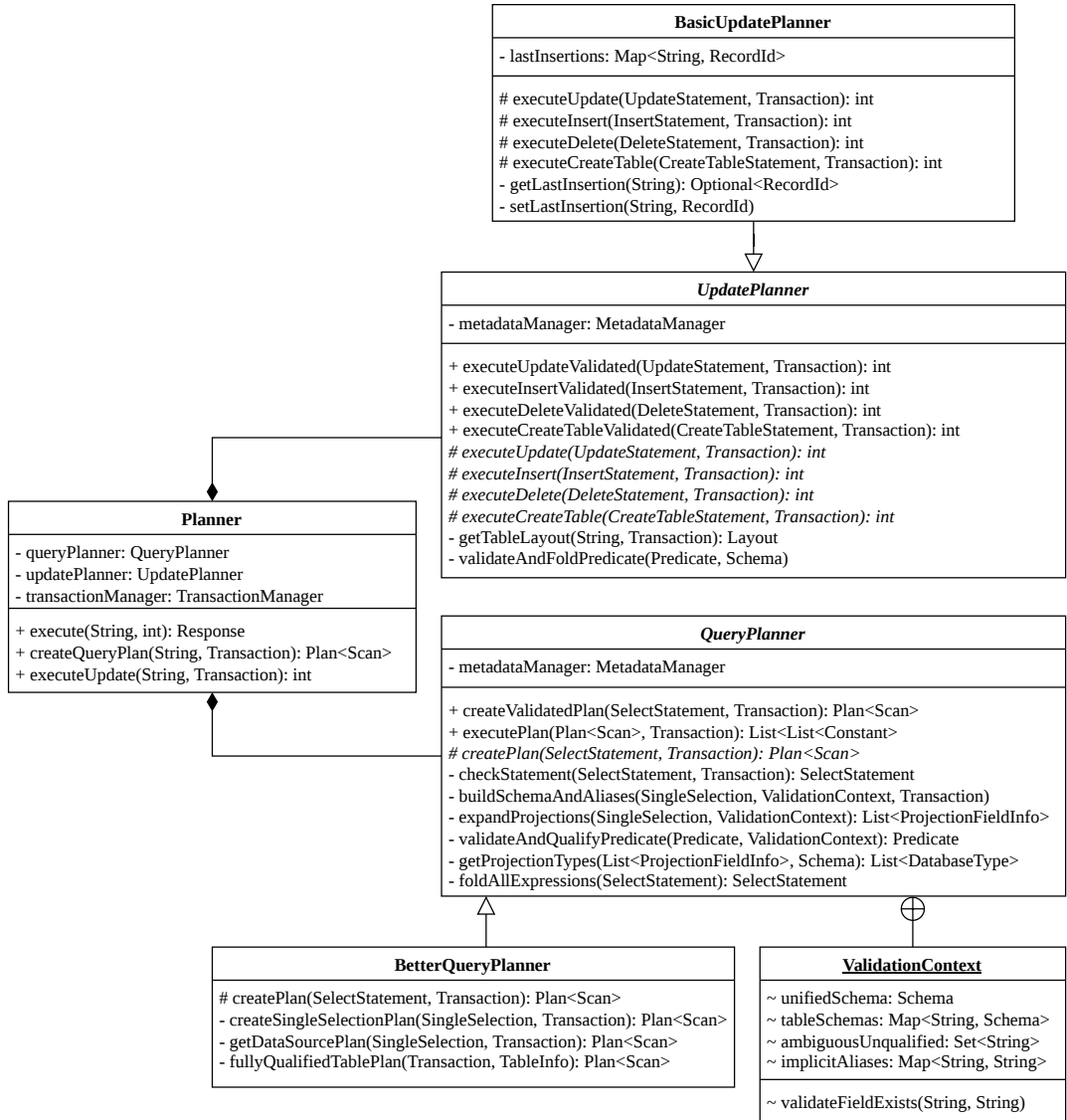
- *createQueryPlan* i *executeUpdate* koje su prilagođene *JDBC* (*Java Database Connectivity*) *API* interfejsu. *JDBC* definiše generičko ponašanje za interakciju sa

²<https://www.postgresql.org/docs/current/planner-optimizer.html>

³<https://www.postgresql.org/docs/current/planner-stats-details.html>

sistemima za upravljanje bazama podataka (ne postoji konkretna implementacija za LBDB, ali definisanjem ovih metoda ju je lako dodati). `createQueryPlan` kreira plan za *read-only* naredbu, ali ga ne izvršava, dok se `executeUpdate` oslanja na to da su modifikacione naredbe dizajnirane da se odmah izvrše i vraća broj promenjenih slogova,

- `execute` koja je prilagođena **klijentsko serverskoj arhitekturi** LBDB sistema, u okviru koje se brine o automatskom ili manuelnom potvrđivanju transakcija, kreiranju i izvršavanju plana. Vraća neki **Response** objekat, koji enkapsulira sve moguće vrste odgovora na neku naredbu.



Slika 20: Struktura planera

7.2.3 Planiranje *read-only* naredbi

Kao što je već spominjano u tekstu, SQL naredbe se dele na *read-only* i modifikacione. Glavni primer *read-only* naredbe je SELECT naredba, koja služi za struktuirano upitivanje (eng. *query*) baze podataka.

Svaka SELECT naredba prvo mora proći semantičku proveru pre pravljenja sâmog plana. Semantička provera se sastoji od sledećih koraka:

- provera postojanja fizičkih tabela spomenutih u naredbi
- proširenje zamenskih članova na konkretne kolone
- provera da se zamenski članovi ne koriste u izrazima
- provera postojanja kolona pomenutih u izrazima i predikatu
- provera dvosmislenih imena kolona (u slučaju da dve tabele imaju isti naziv kolone i ne može da se trivijalno skonta koja se koristi)
- provera da li aritmetičke operacije mogu da se izvrše za tip kolone
- provera da li kolone u unijama imaju iste tipove

QueryPlanner apstraktna klasa pruža implementaciju semantičke provere, a konkretni algoritmi planiranja SELECT naredbe koji je nasleđuju mogu da podrazumevaju da su naredbe koje dobiju sigurno semantički validne. QueryPlanner takođe redukuje sve izraze i predikat pomoću PartialEvaluator klase.

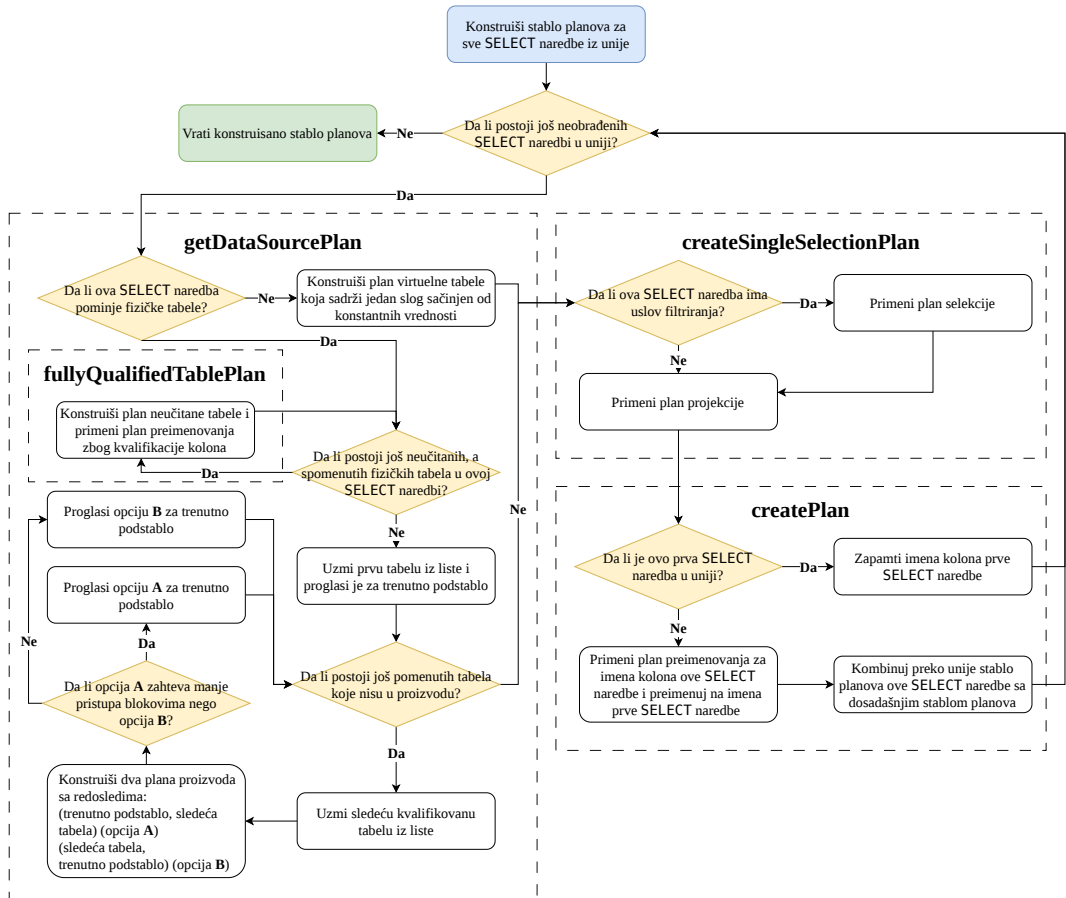
7.2.3.1 Algoritam planiranja SELECT naredbi

BetterQueryPlanner klasa nasleđuje QueryPlanner i predstavlja implementaciju osnovnog planera koji podržava sve alternative SELECT naredbe predstavljene u [njenoj gramatici](#).

Stablo relacionih operatora je korektno (eng. *sound*), ako svi slogovi koje ono proizvodi ispunjavaju sve uslove relacionih operacija definisanih nekom SELECT naredbom. Stablo planova, koje se prevodi u stablo relacionih operatora, koje BetterQueryPlanner konstruiše je uvek korektno.

Problem BetterQueryPlanner implementacije je niska efikasnost konstruisanih stabala planova, jer ne koristi ni tehnike *RBO* planiranja, ni tehnike *CBO* planiranja, već izvršava samo minimalni skup koraka koji su neophodni da se obezbedi korektnost.

Algoritam planiranja se može videti na [dijagramu ispod](#).



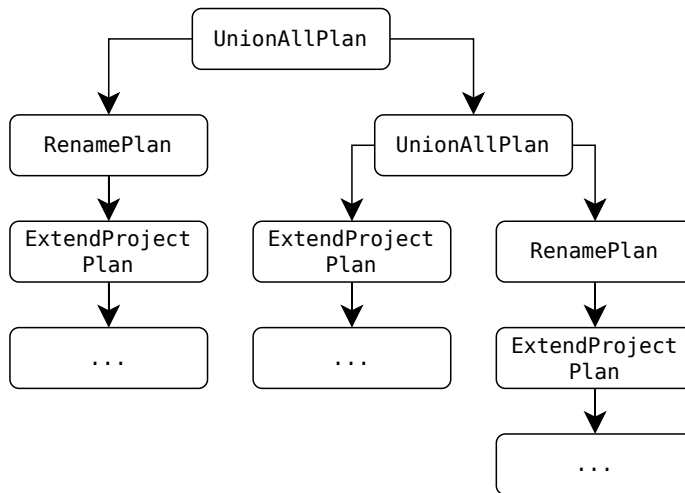
Slika 21: Flowchart dijagram BetterQueryPlanner algoritma planiranja

Kreira finalno stablo planova kroz četiri funkcije koje zovu jedne druge, imaju rastući prioritet i zadužene su za različite operacije:

- `createPlan` funkcija je ulazna tačka algoritma, definisana u `QueryPlanner` klasi i ima zaduženje kreiranja unija više `SELECT` naredbi, u slučaju postojanja `UNION ALL` gramatičke alternative.

Unije `SELECT` naredbi zahtevaju da se kolonama svake `SELECT` naredbe pristupa pomoću imena datih u prvoj `SELECT` naredbi. Zbog ovoga se dodaje operator preimenovanja ispred svakog podstabla planova svake naredbe u uniji, sem prve. Operacija unije ima najmanji prioritet, pa se poslednja izvršava.

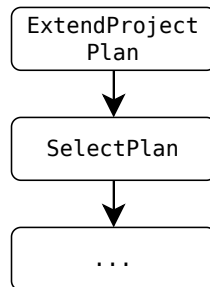
Ukoliko ne postoji `UNION ALL` gramatička alternativa, funkcija vraća podstablo planova jedine `SELECT` naredbe.



Slika 22: Primer stabla planova nakon unije 3 naredbe

- `createSingleSelectionPlan` funkcija obezbeđuje korektnost filtriranja i projekcije.

Dodaje operator filtriranja samo ako filter postoji; dodaje nove virtuelne kolone i briše kolone koje nisu spomenute.



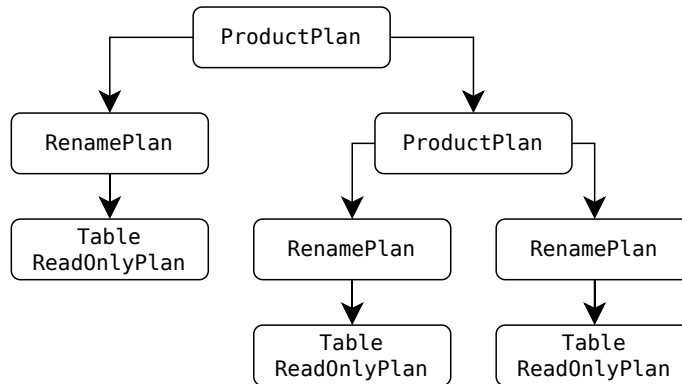
Slika 23: Primer stabla planova nakon filtriranja i projekcije

- `getDataSourcePlan` funkcija se brine o tome odakle će doći slogovi i ima dve putanje izvršavanja.

Prva putanja izvršavanja se dešava kada se u `SELECT` naredbi ne spominje ni jedna fizička tabela, već se radi upit virtuelne tabele koja ima jedan slog koji se sastoji samo od konstanti. U tom slučaju, samo vraća jedan jedini `DummyTablePlan` plan čvor.

Druga putanja izvršavanja se dešava kada se spominje jedna ili više fizičkih tabela. Ako se spominje jedna fizička tabela, njen plan biva vraćen. Ako se spominje više od jedne fizičke tabele, potrebno je uraditi operaciju proizvoda. Proizvod tabela se vrši tako što se prva spomenuta tabela proglasi da bude početna, pa se prolazi kroz sve ostale spomenute tabele i ponavlja se postupak: kreiraju se dva proizvod plana, jedan gde je dosadašnje podstablo planova na levom mestu, a plan sledeće tabele na desnom i jedan gde je redosled obrnut;

plan koji ima manje pristupa blokovima se uzima kao sledeći koren podstabla planova i postupak se izvršava dok se ne prođe kroz sve pomenute tabele. Time se dobija oformljeno podstablo planova gde su proizvodi tabele zadovoljeni. Ovakva provera broja pristupanih blokova nije optimalna, ali može pomoći u otklanjanju veoma neefikasnih stabala planova i predstavlja jedino mesto gde se primenjuje *RBO* tehnika planiranja.



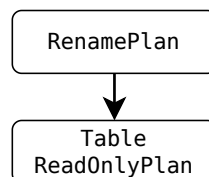
Slika 24: Primer stabla planova nakon proizvoda 3 tabele

- `fullyQualifiedTablePlan` obezbeđuje davanje punokvalifikujućih imena kolonama fizičkih tabela.

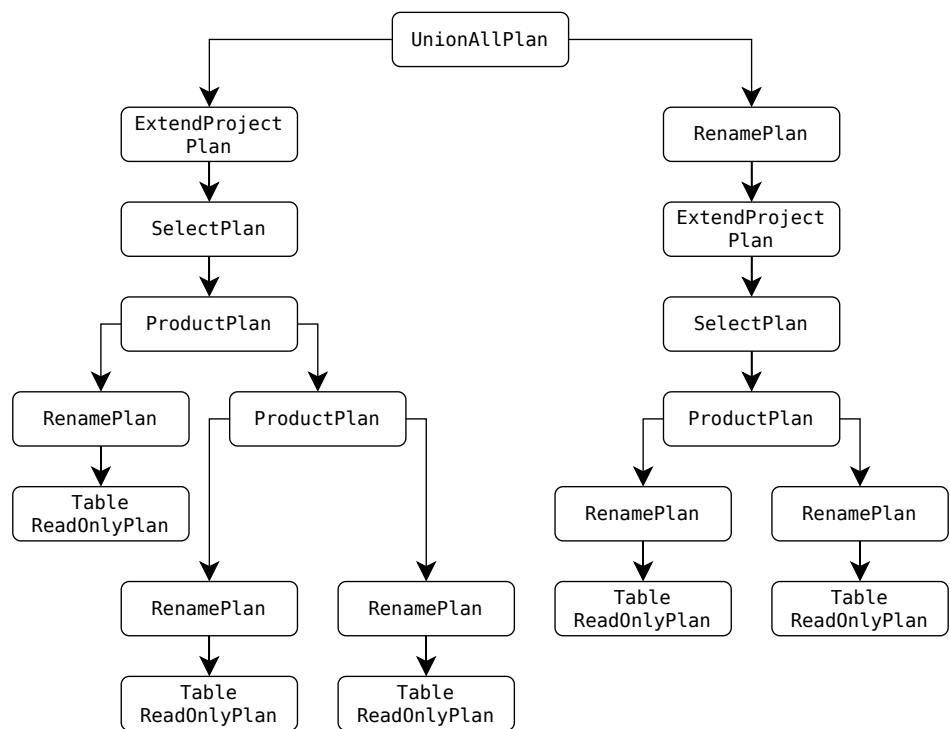
U slučajevima gde se vrši proizvod dve (ili više) tabele, postoji mogućnost da te dve tabele imaju istoimenu kolonu. Ovo je čest slučaj jer povećava čitljivost upita i strukture tabela, pa je za njega potrebno pružiti adekvatnu podršku.

U prethodnoj funkciji koja vrši proizvode, rečeno je da se direktno barata sa planovima tabela. Ovo nije precizno, jer se ne barata direktno sa tabelama, već sa podstablom planova koje predstavlja tabelu sa punokvalifikovanim imenama kolona. Kvalifikacija imena kolona se vrši preko operatora preimenovanja, tako što se pre samog imena kolone doda ime tabele i tačka (`ime_kolone` postaje `ime_tabele.ime_kolone`). Dozvoljava i preimenovanje tabela u slučaju da vršimo proizvod dve (ili više) iste tabele.

Ako se proizvod radi samo sa punokvalifikovanim tabelama, ne postoji mogućnost da sistem ne može da prepozna kojoj tabeli kolona pripada, sem ako korisnik nije zadao semantički neispravnu naredbu.



Slika 25: Primer stabla planova nakon kvalifikacije kolona tabele



Slika 26: Primer kompletnog stabla planova

7.2.3.2 Automatsko generisanje opisa planova

Drugi primer *read-only* naredbe je EXPLAIN naredba. Njena uloga u sistemu je tabelarno ispisivanje kompletnih stabla planova. EXPLAIN naredba se poziva tako što se doda ključna reč EXPLAIN ispred SELECT naredbe. Nije podržana za modifikacione naredbe.

EXPLAIN naredba je implementirana tako da generiše slogove koji prate šemu specijalne tabele koja ima sledeće kolone: ime relacionog operatora, procena kroz koliko blokova će taj relacioni operator proći da generiše sve slogove, procena broja slogova i specijalni detalji. Svaki slog predstavlja čvor rezultujućeg stabla relacionih operatora. Iako je poenta naredbe tabelarni prikaz stabla, naredba samo generiše ove slogove i ne brine se o [formatiranju tabele](#).

Scan	Block est.	Record est.	Details
ExtendProjectScan	5	52	
└ SelectScan	5	52	student.isactive = TRUE
└└ RenameScan	5	103	
└└└ TableScan	5	103	'student'

Listing 8: Primer rezultujuće tabele EXPLAIN naredbe

7.2.4 Planiranje modifikacionih naredbi

Modifikacione naredbe su razne, a UpdatePlanner ima istu ulogu za njih, kao što QueryPlanner ima za SELECT naredbu, a to je samo semantička provera. Konkretni algoritmi planiranja modifikacionih naredbi mogu da podrazumevaju da je naredba semantički validna i da su izrazi i predikati redukovani pomoću PartialEvaluator klase. Za svaku modifikacionu naredbu su opisani koraci za semantičku proveru.

INSERT naredba služi za umetanje novih slogova. Semantička provera INSERT naredbi se sastoji od sledećih koraka:

- provera postojanja fizičke tabele u koju se umeću novi slogovi,
- provera broja kolona novih slogova,
- provera tipova kolona novih slogova sa tipovima kolona definisanih u šemi tabele,
- provera dozvole *NULL* vrednosti za kolone gde je vrednost novih slogova *NULL*.

UPDATE naredba služi za ažuriranje vrednosti postojećih slogova na osnovu nekog uslova filtriranja. Semantička provera UPDATE naredbi se sastoji od sledećih koraka:

- provera postojanja fizičke tabele čiji se slogovi ažuriraju,
- provera postojanja kolona pomenutih u predikatu i izrazima ažuriranja,
- provera dozvole *NULL* vrednosti za kolone gde je nova vrednost *NULL*,
- provera tipova kolona ažuriranih slogova sa tipovima kolona definisanih u šemi tabele.

DELETE naredba služi za brisanje postojećih slogova na osnovu nekog uslova filtriranja. Semantička provera DELETE naredbi se sastoji od sledećih koraka:

- provera postojanja fizičke tabele čiji se slogovi brišu,
- provera postojanja kolona pomenutih u predikatu.

CREATE TABLE naredba služi za kreiranje novih tabela. Semantička provera CREATE TABLE naredbi se sastoji od sledećih koraka:

- provera da li tabela sa tim imenom već postoji,
- provera da li je veličina sloga prevazilazi maksimalnu veličinu sloga. **TODO: citirati limitaciju unspanned sloga**

7.2.4.1 Algoritam planiranja INSERT naredbe

Stablo planova za INSERT naredbe je uvek isto i sastoji se samo od jednog TablePlan čvora. Taj čvor se pretvara u svoj prateći relacioni operator nad kojim se vrše umetanja novih slogova. Vraća broj dodatih slogova.

Algoritam umetanja novog sloga implementiran u TableScan operatoru funkcioniše tako što traži prvo slobodno mesto za nov slog, ali počevši od pozicije trenutnog sloga tog TableScan objekta. Nakon što se operator inicijalizuje, pozicioniran je na početku tabele, to jest pre prvog sloga. Ovo znači da će umetanje prvog sloga u listi novih slogova uvek počinjati od početka. Prednost ovog pristupa je to što će obrisani slogovi brzo biti ponovo popunjeni, pa se prostor maksimalno dobro iskorišćava. Mana ovog pristupa je to što umetanje prvog

novog sloga može da potraje, jer u najgorem slučaju mora da se prođe kroz sve slogove tabele da se pronađe prazno mesto. Drugi način implementacije algoritma je da se umetanje novih slogova uvek vrši od kraja. Prednost je konzistentno dobra brzina umetanja, jer se preskače pretraga za slobodno mesto. Mana je to što se sve više i više prostora baca na obrisane slogove.

Implementirano rešenje je kompromis ova dva algoritma, gde se za svaku tabelu pamti pozicija poslednje umetnutog sloga i novi slogovi se umeću od te pozicije. Pamćenje pozicija poslednje umetnutih slogova važi samo dok je sistem upaljen i resetuje se prilikom gašenja sistema. Zadržava prednost brzine umetanja, a nakon restarta sistema mesta obrisanih slogova mogu ponovo biti popunjena.

7.2.4.2 Algoritam planiranja UPDATE naredbe

Stablo planova za UPDATE naredbe se može sastojati samo od jednog `TablePlan` čvora, ali ispred njega može stojati i `SelectReadOnlyPlan` čvor u slučaju da slogovi koji trebaju biti ažurirani moraju da ispune neki uslov filtriranja. Ova dva (ili jedan) čvora se pretvaraju u svoje prateće relacione operatore koji znaju da postavе nove vrednosti. Vraća broj ažuriranih slogova.

Za razliku od INSERT naredbe, UPDATE naredba može da sadrži izraze koji nisu konstantni, to jest koji pominju kolone tabele koja se ažurira.

7.2.4.3 Algoritam planiranja DELETE naredbe

Stablo planova za DELETE naredbe se može sastojati samo od jednog `TablePlan` čvora, ali ispred njega može stojati i `SelectReadOnlyPlan` čvor u slučaju da ne trebaju da se obrišu svi slogovi iz tabele već samo oni koji ispunjavaju uslov filtriranja. Vraća broj obrisanih slogova.

Brisanje nekog sloga samo označava mesto gde se taj slog nalazio kao slobodno za umetanje novog sloga, umesto da radi kompresiju datoteke i zapravo izvrši fizičko brisanje.

7.2.4.4 Algoritam planiranja CREATE TABLE naredbe

CREATE TABLE naredba je specijalna vrsta naredbe jer ne modifikuje slogove običnih tabela, već slogove [kataloških tabela](#). Dodaje jedan slog u katalošku tabelu koja pamti sve postojeće tabele. Dodaje slog za svaku kolonu nove tabele u katalošku tabelu koja pamti sve postojeće kolone. Pošto ne utiče na slogove običnih tabela, uvek vraća nula za broj slogova na koje je uticala.

Menadžer metapodataka tabela interno konstruiše tri `TableScan` objekta: prvi koristi da proverí jedinstvenost imena nove tabele, drugi koristi da umetne slog za novu tabelu u `tablecatalog` katalošku tabelu, a treći koristi da umetne slogove novih kolona u `fieldcatalog` katalošku tabelu. Posledica ovakve implementacije je ta da `UpdatePlanner`

apstraktna klasa zapravo ne vrši semantičku proveru pre pozivanja algoritma planiranja CREATE TABLE naredbe, nego reaguje na greške i transformiše ih, ako se dese.

Glava 8

Klijentsko serverska arhitektura

Postoje dva glavna načina kako sistem upravljanja sistema relacionih baza podataka može raditi: lokalno, bez mrežne infrastrukture (eng. *embedded connection*) i kao server.

Karakteristike sistema u lokalnom režimu rada:

- radi u istom procesu operativnog sistema kao i aplikacija koja ga koristi,
- samo jedna aplikacija može da koristi tu instancu sistema,
- ne zahteva mrežnu konekciju,
- zahteva manje resursa,
- ponaša se kao biblioteka koja ponudjuje unutrašnjosti relacionog modela podataka.

Karakteristike sistema u serverskom režimu rada:

- radi kao zaseban proces operativnog sistema,
- više različitih aplikacija i korisnika može da pristupi toj instanci sistema,
- zahteva mrežnu konekciju,
- zahteva više resursa,
- ponaša se kao pružilac usluge baratanja relacionim modelom podataka.

LBDB sistem podržava samo serverski režim rada.

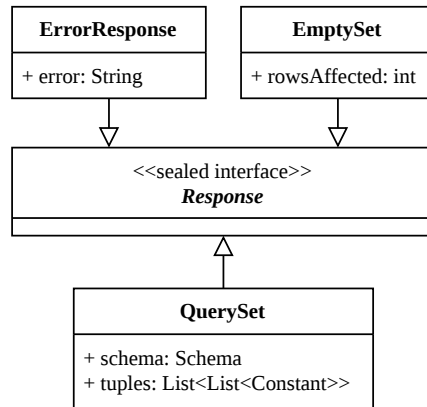
8.1 Komunikacija sa klijentima

Kod serverskog režima rada, predpostavlja se da je klijent na udaljenom računaru i nema pristup nikakvim resursima računara na kom se pokreće server. Posledica ovog je da sva komunikacija mora da se vrši kroz mrežu, preko nekog protokola komunikacije.

8.1.1 Odgovori sistema

Sve vrste odgovora koje server može dati za neku naredbu koju je klijent zadao predstavljene su Java *record* konstruktom i implementiraju *Response sealed interface*.

- *QuerySet* predstavlja odgovor na *read-only* naredbe, čiji je rezultat skup slogova. Jedan slog je predstavljen listom konstanti. Da bi sistem znao kako da pošalje slogove preko mreže, a kasnije i kako da napravi tabelarni prikaz, potrebno je poslati i propratnu šemu koja opisuje poslate slogove.
- *EmptySet* predstavlja odgovor na modifikacione naredbe. Sadrži samo podatak o tome na koliko slogova je uticano.
- *ErrorResponse* se vraća klijentu kada se desi greška prilikom parsiranja, prilikom planiranja ili prilikom izvršavanja naredbe (deljenje sa nulom, ...).



Slika 27: Struktura Response hijerarhije

8.1.2 Protokol serijalizacije

Preko mreže je moguće slati samo niz bajtova. Pošto Response objekti nisu podrazumevano predstavljeni nizovima bajtova, za njih se mora definisati način serijalizacije i deserijalizacije u i iz niza bajtova.

Protokol serijalizacije Response objekata je inspirisan *RESP*⁴ protokolom. Svakom tipu se dodeljuje jedan bajt koji ga jedinstveno predstavlja. '*' za QuerySet, '_' za EmptySet i '-' za ErrorResponse.

Brojčane vrednosti se pretvaraju u bajtove byte tipa koji se sastoji od jednog bajta, u bajtove short tipa koji se sastoji od 2 bajta ili u bajtove int tipa koji se sastoji od 4 bajta. String vrednosti se pretvaraju u bajtove preko UTF-8 kodiranja.

EmptySet i ErrorResponse je trivijalno serijalizovati i deserijalizovati, jer se sastoje od samo jedne vrednosti.

QuerySet sadrži dosta vrednosti koje imaju specifičan format, pa je algoritam serijalizacije kompleksniji (svaka vrednost je podrazumevano zapisana u bajtovima):

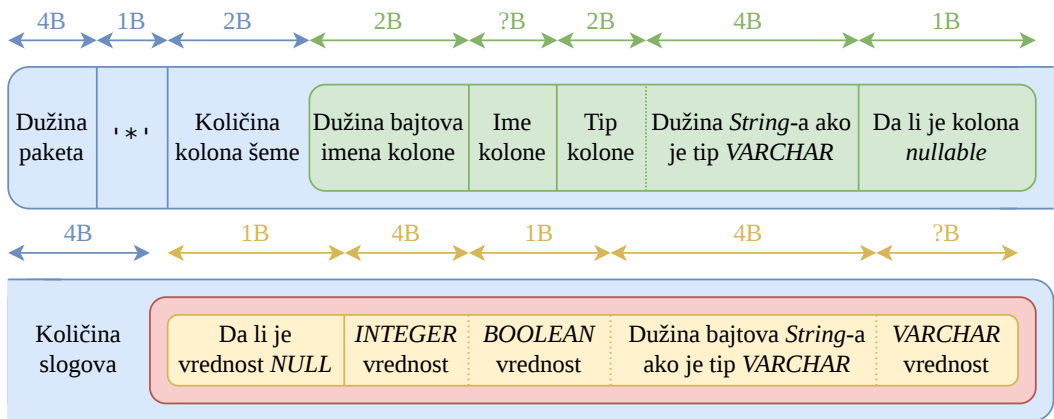
- zapisuje se jedinstveni identifikator QuerySet tipa: '*'
- zapisuje se količina kolona šeme rezultujuće tabele kao short
- za svaku kolonu se zapisuje:
 - dužina bajtova imena kao short
 - ime kolone
 - tip kolone kao short, ako je tip *VARCHAR* zapisuje se i njegova dužina kao int
 - da li je kolona *nullable*, kao byte
- zapisuje se količina slogova
- za svaku vrednost svakog sloga se zapisuje:

⁴<https://redis.io/docs/latest/develop/reference/protocol-spec/>

- ▶ 0 ako je *NULL*, 1 ako nije *NULL* kao short
- ▶ brojnu vrednost kao *int* ako je tip *INTEGER*, brojnu vrednost kao *byte* ako je tip *BOOLEAN*, dužina *String*-a kao *int* tip zajedno sa kodiranim *String*-om ako je tip *VARCHAR*

Na kraju serijalizacije se računa dužina bajtova i ona se stavlja pre svih bajtova, da bi se prilikom deserijalizacije znalo koliko bajtova da se pročita.

Algoritam deserijalizacije za sve tipove isto funkcioniše, ali u suprotnom smeru.



Slika 28: Struktura QuerySet paketa

Na slici plavo predstavlja ceo paket, zeleno predstavlja deo paketa koji se ponavlja za svaku kolonu šeme, crveno predstavlja deo paketa koji se ponavlja za svaki slog, a žuto predstavlja deo paketa koji se ponavlja za svaku vrednost sloga.

8.2 Serverski sloj

Klasa *LBDBServer* sadrži main funkciju serverske aplikacije sistema. Čita i validira argumente komandne linije: port na kom će server raditi i putanja gde će se čuvati datoteke sistema. Pokreće instancu servera sa prosleđenim argumentima. Sva logika koja podržava serverske operacije se nalazi u *Server* klasi.

8.2.1 Rukovođenje klijentima

Prva funkcionalnost servera je obrada klijentskih konekcija i naredba koje one šalju. Prilikom pokretanja servera, instancira se serverski soket koji prihvata konekcije na određenom portu i otvara klijentski soket za svakog klijenta.

Pošto su klijenti nezavisni, ne bi trebalo da čekaju jedni na druge i zbog toga se svaki obrađuje u zasebnoj niti. Zarad lakšeg upravljanja nitima, postoji *ThreadPool* sa 8 fiksnih, stalno postojećih niti koje čim završe sa obradom jednog zahteva ponovo postaju slobodne.

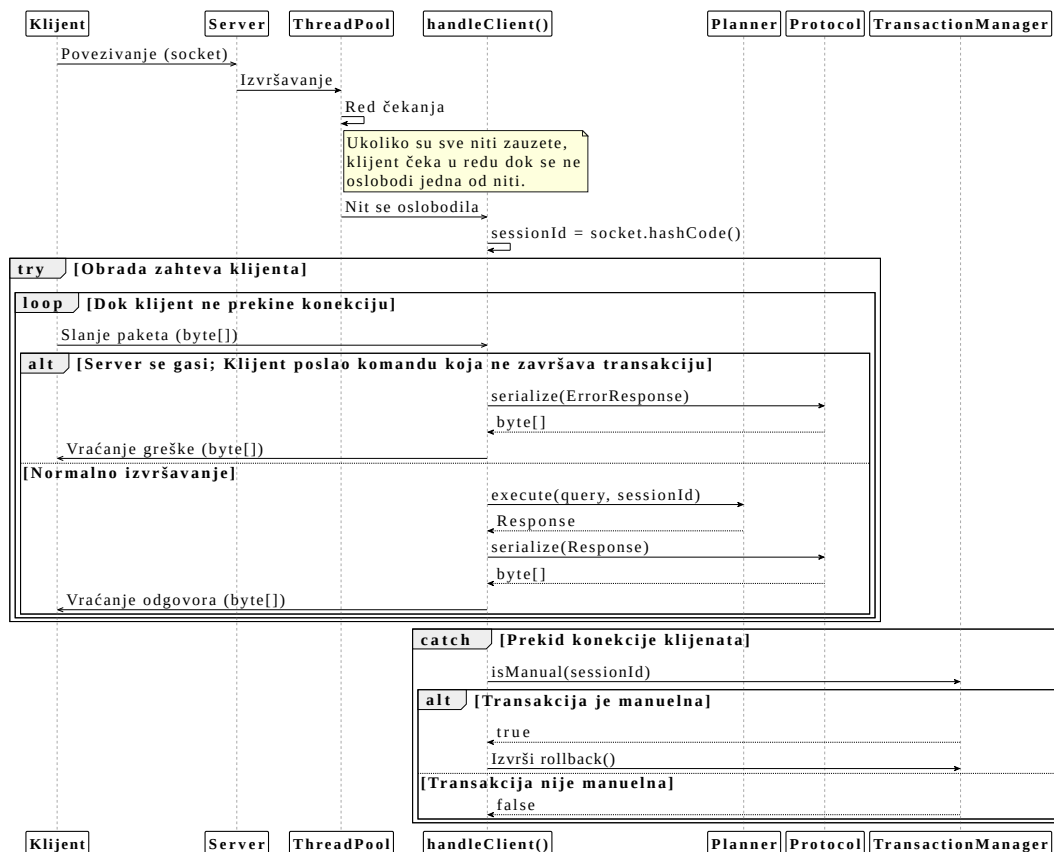
8.2.1.1 Obrada jednog klijenta

Obrada zahteva klijenata se vrši kroz `handleClient()` funkciju. Svaki klijentski soket se kodira u jedinstveni broj sesije, tako što se uzme heš vrednost njegove konekcije. Ovime se omogućava mapiranje klijenta na njegovu trenutnu transakciju. Ako broj sesije klijenta ne postoji u sistemu, dodeljuje se nova (prva) transakcija za tog klijenta.

Kao što je već spomenuto, klijentske transakcije mogu da rade u režimu gde se sastoje od jedne naredbe (*autocommit*) ili u režimu gde se sastoje od više naredbi. U režimu gde se transakcije sastoje od više naredbi, potrebno je početi ih sa `START TRANSACTION` naredbom, a završiti sa `COMMIT` ili `ROLLBACK` naredbama. Ovo je glavni razlog zašto je potrebno jedinstveno identifikovati sesije klijenata, da bi njihova transakcija mogla da perzistira kroz više naredbi.

U slučaju prekida konekcije, server će izvršiti `ROLLBACK` trenutne transakcije klijenta.

U slučaju da se server gasi, povezani klijenti mogu da pošalju samo naredbe koje završavaju transakcije, ali više o tome u opisu gašenja servera.



Slika 29: Dijagram sekvence obrade klijenta

8.2.2 Pisanje kontrolnih tačaka

Druga funkcionalnost servera je određivanje kada (ali ne i kako) će mirna kontrolna tačka biti pisana. Prilikom pokretanja servera se startuje i nit koja na svakih 10 minuta započinje pisanje mirne kontrolne tačke. [Kao što je već napomenuto](#), da bi se mirna kontrolna tačka zapisala, ni jedna transakcija ne sme biti aktivna u sistemu.

Kada prođe 10 minuta od poslednjeg zapisa mirne kontrolne tačke, server poziva algoritam zapisa koji interno čeka da se sve transakcije završe i zaustavlja obradu novih.

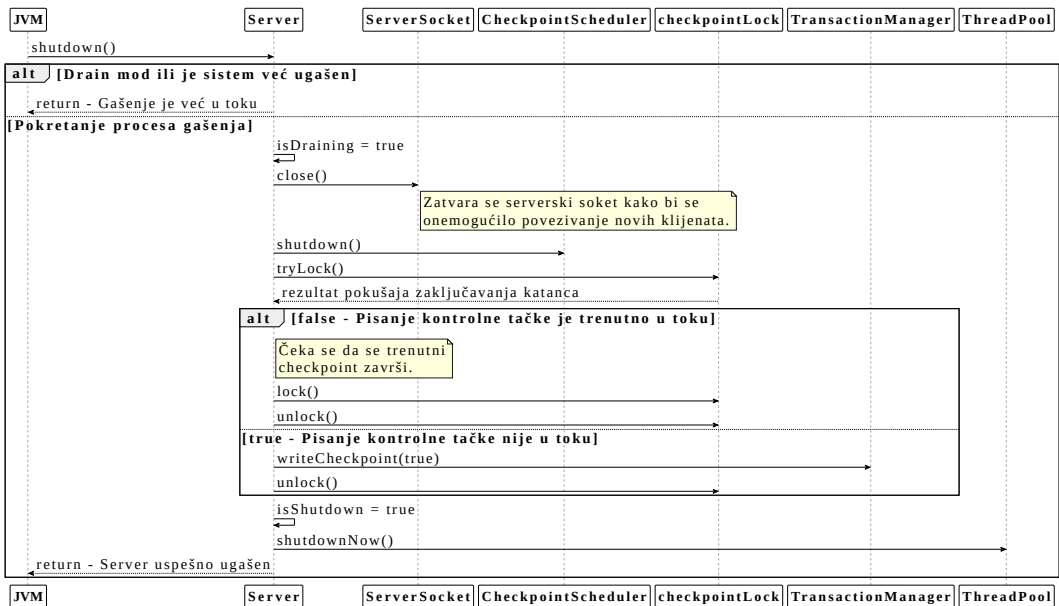
8.2.3 Gašenje sistema

Treća funkcionalnost je bezbedno gašenje sistema. Bezbedno gašenje se inicira slanjem SIGINT signala na Unix operativnim sistemima ili slanjem CTRL_C (ili sličnog) signala na Windows operativnom sistemu. Najčešće, ovo se mapira na gašenje prozora gde je server pokrenut, ili rađenjem CTRL + C prečice.

Bezbedno gašenje se sastoji iz tri koraka:

- prestajanje prihvatanja novih naredbi, sem naredbi završetka transakcije,
- čekanje da se završe sve transakcije,
- zapisivanje mirne kontrolne tačke.

Nakon što je gašenje inicirano, ulazi se u drain mod, gde se ne prihvataju konekcije novih klijenata, ali starim klijentima je dozvoljeno da završe svoje transakcije preko COMMIT ili ROLLBACK naredbi. U drain modu, samo ove naredbe su dozvoljene. Ako je server započeo pisanje mirne kontrolne tačke kada je zahtev za gašenje pokrenut, neće se pisati još jedna.



Slika 30: Dijagram sekvence bezbednog gašenja servera

8.3 Klijentski sloj

Klasa `LBDBClient` sadrži `main` funkciju klijentske aplikacije sistema. Čita i validira argument komandne linije: port na kojem se nalazi server na koji se klijent povezuje. Sva logika slanja naredbi se nalazi u ovoj klasi.

Klijentska aplikacija pruža korisnicima terminal gde se naredbe mogu upisivati. Terminal podržava automatsko završavanje ključnih reči (eng. *auto complete*) pritiskom TAB dirke i navigaciju istorije komandi. Implementaciju ovih stvari podržava **TODO**: ``jline`` zavisnost. Terminal prati i koliko vremena se izvršavala svaka naredba.

Paketi koje šalje serveru su serijalizovani tako da prvo stoji dužina teksta naredbe u bajtovima, a zatim i kodirani tekst naredbe. Pakete koje prima od servera deserijalizuje tako što prvo čita prva 4 bajta koja predstavljaju dužinu paketa u bajtovima, a zatim koristi algoritam deserijalizacije koji je opisan u [protokolu](#).

Podržava ispis sve tri vrste `Response` objekata, gde se `ErrorResponse` i `EmptySet` trivijalno prikazuju jer sadrže samo jednu vrednost. `QuerySet` objekti su tabele i njihovo prikazivanje se radi algoritmom štampanja tabela.

8.3.1 Štampanje tabela

Algoritam štampanja tabela ima zadatak da lepo formatira sva imena kolona i sve vrednosti svih slogova iz `QuerySet` objekta. U tom objektu stoji šema tabele, pa ovo nije zahtevan zadatak. Koristi `StringBuilder` za efikasno kreiranje *String*-ova. Za lep prikaz koristi specijalne *Unicode* karaktere kao što su: `┌`, `─`, `┐`, `└`, `├`, `│`, `┬`, `┴`, `┤`, `┘`.

tableid	tablename	slotsize
2	tablecatalog	112
3	fieldcatalog	121
5	department	164
6	professor	173
7	student	173
8	course	172
9	enrollment	20

Listing 9: Primer ispisane tabele `SELECT * FROM tablecatalog`; naredbe

8.3.2 Masovno pokretanje naredbi

LBDB paket pruža još jednu vrstu klijentske aplikacije: `BulkExecutor`. Ova klijentska aplikacija funkcioniše slično kao i obična klijentska aplikacija, ali umesto pružanja interakcije sa sistemom preko terminala, redom izvršava sve SQL naredbe iz neke datoteke. Ovo radi u manuelno započetoj transakciji i ako bar jedna naredba ne uspe sa izvršavanjem, javlja grešku i vrši *rollback*. Korisna je za popunjavanje tabela ili za testiranje sistema.

Pošto je za korektno funkcionisanje sistema potrebno mnogo kompleksnih funkcionalnosti i algoritama, potrebno je izvršiti intenzivno testiranje istih da bi se dokazala pravilna implementacija. Slojevi od kojih se sistem sastoji su testirani izolovano, sa time da se slojevi na višim apstrakcionim nivoima oslanjaju na slojeve na nižim apstrakcionim nivoima.

9.1 Organizacija testova

Sve testove u sistemu podržava *JUnit* biblioteka. *JUnit* sadrži razne konfiguracione parametre, a za *LBDB* sistem testiranja su najbitniji parametri koji omogućavaju [definisanje čistača](#) i parametri koji omogućavaju paralelno pokretanje testova.

```
junit.jupiter.extensions.autodetection.enabled = true
junit.jupiter.execution.parallel.enabled = true
junit.jupiter.execution.parallel.mode.default = concurrent
```

Listing 10: Konfiguracioni parametri *JUnit* biblioteke

Sistem prati standardnu definiciju strukture direktorijuma izvornog koda *Maven* sistema za upravljanje zavisnostima. Više o njemu u [pregledu sistema](#). Po *Maven*-u, testovi se nalaze unutar `src/test/java` direktorijuma, a konfiguracioni parametri *JUnit* biblioteke se nalaze unutar `src/test/resources` direktorijuma. Testovi su grupisani po istim modulima kao i glavni izvorni kod.

9.1.1 Testno okruženje

Da bi se postiglo korektno i unifikovano testiranje svih funkcionalnosti, potrebno je pružiti im odgovarajuće testno okruženje. Pošto većina funkcionalnosti zahteva rad sa datotekama, glavna dužnost testnog okruženja je da izoluje direktorijume gde će se ove datoteke nalaziti. Time se postiže da test *A* koji kreira na primer tri tabele ne može da utiče na test *B* koji kreira dve tabele gde se imena tabela poklapaju.

`TestUtils` je pomoćna klasa koja pruža implementaciju ove izolacije, ali pruža i dodatne pomoćne metode koje olakšavaju testiranje:

- provera postojanja datoteka,
- dobavljanje privatnih polja putem Java refleksije.

Za podsistem relacionih operatora, postoji pomoćna klasa `QueryTestUtils` koja pruža dodatne pomoćne metode za testiranje ovog podsistema:

- inicijalizacija i popunjavanje jedne tabele sa 250 slogova, koja ima tri *Integer*, tri *String* i tri *Boolean* kolone,

- inicijalizacija i popunjavanje dve tabele sa po 250 slogova koje imaju istu šemu kao tabela u prethodnoj pomoćnoj metodi.

Za podsistem planiranja, postoji pomoćna klasa `PlanTestUtils` koja pruža dodatne pomoćne metode za testiranje ovog podsistema:

- pokretanje **samo** provere `SELECT` naredbe i vraćanje proširenog `SelectStatement` objekta,
- pokretanje **samo** provere modifikacionih naredbi,
- pravljenje stabla planova za `SELECT` naredbe,
- pokretanje modifikacionih naredbi,
- kreiranje novih transakcija, korisno za proveru izolacije,
- inicijalizacija tri prazne ili tri popunjene tabele, gde prve dve imaju istu šemu kao kod pomoćne klase za testiranje podsistema relacionih operatora, a treća sadrži samo jedno `Integer` polje i korisna je za testiranje spajanja tabela samih sa sobom.

9.1.2 Sistem izolacije direktorijuma na disku

Prvi od dva načina pokretanja testova je u okviru direktorijuma koji se nalaze na fizičkom disku. Prednosti ovog načina pokretanja su laki pregled generisanih datoteka zarad otklanjanja grešaka i nezahtevno pokretanje. Mana ovog načina pokretanja je brzina jer je pristup fizičkom disku spor.

Da bi se postigla izolacija testova i kroz iteracije pokretanja istih testova, potrebno je očistiti stare direktorijume. `JUnit` omogućava konfiguraciju čistača, to jest funkcije koja se izvršava pre svih testova. `GlobalCleanup` klasa sadrži ovu logiku.

9.1.3 Sistem izolacije direktorijuma u radnoj memoriji

Drugi od dva načina pokretanja testova je u okviru direktorijuma koji se nalaze u radnoj memoriji. Pošto je `LBDB` sistem kompatibilan sa `java.nio.file API`-jem, rukovođenje direktorijumima u radnoj memoriji se vrši preko `Jimfs` biblioteke. Prednost ovog načina pokretanja je brzina testova. Mane ovog načina pokretanja su težak pristup datotekama zarad otklanjanja grešaka i velika potrošnja radne memorije.

U okviru `TestUtils` klase se podešava način pokretanja testova, gde je pokretanje u radnoj memoriji podrazumevano podešeno. `TestUtils` definiše jednu instancu `Jimfs` implementacije `Filesystem` klase koja se koristi za sve testove. Nema potrebe za čišćenjem preko `GlobalCleanup` klase jer se radna memorija sama čisti kada se proces u kom su pokrenuti testovi završi.

Glava 10

Pregled sistema

U okviru ovog poglavlja su objašnjeni raznovrsni detalji sistema koji nisu vezani za sâme funkcionalnosti sistema.

10.1 Izgradnja i pokretanje

Sistem koristi *Maven*⁵ za: automatizaciju kompilacije, izgradnju artifakata (aplikacija koje se pokreću) i za rukovođenje zavisnostima. U *Maven* ekosistemu, izvorni kod prati striktno definisanu strukturu i nalazi se unutar `src/main` direktorijuma.

Za korektno funkcionisanje *Maven* aplikacija, potrebno je definisati `pom.xml` datoteku u kojoj se nalaze sve neophodne instrukcije potrebne *Maven*-u.

Po instrukcijama `pom.xml` datoteke *LBDB* sistema, klijentske aplikacije i serverska aplikacija se grade odvojeno, u tri različita artifakta. Ovo omogućava jednostavno odvojeno pokretanje. Nakon izgradnje, artifakti se mogu pronaći unutar `target` direktorijuma pod imenima: `LBDBServer.jar`, `LBDBClient.jar` i `BulkExecutor.jar`.

```
mvn clean package -Dmaven.test.skip=true
```

Listing 11: Izgradnja svih artifakata sistema, bez pokretanja testova

```
mvn test
```

Listing 12: Pokretanje svih testova u sistemu

10.1.1 Rukovođenje zavisnostima

Klijentske aplikacije i serverska aplikacija dele kod za:

- protokol komunikacije,
- definiciju svih ključnih reči (zbog *auto complete* funkcionalnosti klijentske aplikacije)
- definiciju konstante zbog dobijanja njene *String* vrednosti zarad ispisa,
- definiciju šeme i tipa vrednosti zbog korektnog ispisa.

Sav ostali kod nije deljen, uključujući i zavisnosti koje isto nisu deljene.

10.1.1.1 Zavisnosti servera

Zavisnosti servera su sledeće:

- `datasketches-java` za Java implementaciju *HyperLogLog* strukture podataka⁶,
- `annotations` pruža dodatne Java anotacije poput `@NotNull`⁷.

⁵<https://maven.apache.org/>

⁶<https://datasketches.apache.org/>

10.1.1.2 Zavisnosti klijenta

Zavisnosti običnog klijenta su sledeće:

- `jline-reader`, `jline-terminal` i `jline-terminal-jna` pružaju implementaciju terminala i omogućavaju sistemski agnostičnu podršku za *UTF-8* ispis⁸,
- `net.java.dev.jna:jna` za pristup nativnim instrukcijama operativnog sistema (isto za lepo formatiranje)⁹.

Zavisnosti `BulkExecutor` klijenta su iste kao i zavisnosti običnog klijenta, sa time da `jline-terminal` nije iskorišćen.

10.1.1.3 Zavisnosti testnog okruženja

Ove zavisnosti se koriste u testnom okruženju i ne ulaze u artefakte:

- `junit-jupiter-engine` i `junit-jupiter-params` za pokretanje i definisanje testova¹⁰,
- `jimfs` je implementacija sistema datoteka u radnoj memoriji¹¹,
- `mockito-core` i `mockito-junit-jupiter` za pravljenje objekata koji imaju praznu implementaciju a neophodni su za pozivanje funkcija i metoda¹².

10.2 Sistemska konfiguracija

U okviru sistema postoji i konfiguraciona klasa `LBDBSettings` preko koje je moguće postaviti parametre izbora algoritama ili vrednosti za određene operacije. Podrazumevane vrednosti su dobar izbor za nenadgledanu inicijalizaciju sistema i mogu se videti u sledećem bloku koda:

```
public class LBDBSettings {
    public boolean UNDO_ONLY_RECOVERY = true;
    public int BLOCK_SIZE = 4096;
    public int BUFFER_POOL_SIZE = 128;
    public BufferStrategy bufferStrategy = BufferStrategy.LRU;
    public String LOG_FILE = "log_file";
    public QueryPlannerType queryPlannerType = QueryPlannerType.BETTER;
    public UpdatePlannerType updatePlannerType = UpdatePlannerType.BASIC;
}
```

Listing 13: Kod sistemske klase `LBDBSettings`

⁷<https://github.com/JetBrains/java-annotations>

⁸<https://github.com/jline/jline3>

⁹<https://github.com/java-native-access/jna>

¹⁰<https://junit.org/>

¹¹<https://github.com/google/jimfs>

¹²<https://github.com/mockito/mockito>

Redom, parametri označavaju:

1. **algoritam oporavke** sistema,
2. veličina jednog bloka u bajtovima gde jedan blok predstavlja najmanju jedinicu interakcije sa diskom,
3. količina bafera sa kojim sistem raspolaže,
4. **algoritam izbora** bafera koji će biti smenjen,
5. putanja do datoteke gde se čuvaju podaci potrebni za oporavak sistema i poništavanje transakcija,
6. implementacija planera za operacije upita; podržana samo BETTER implementacija,
7. implementacija planera za operacije modifikacije; podržana samo BASIC implementacija.

10.3 Integracija sa *GitHub* platformom

*GitHub*¹³ platforma omogućava pokretanje testova (eng. *Continious Integration*, *CI*) i izgradnju aplikacija (eng. *Continious Delivery*, *CD*) u okviru njihovih servera, što omogućava ljudima koji rade na softveru da imaju glavni izvor poverenja na jednom mestu. LBDB sistem iskorištava ovu mogućnost i definiše specijalnu *GitHub* datoteku za *CI*. U okviru nje se definiše *Windows* i *Ubuntu Linux* okruženje za testiranje, testovi se pokreću i rezultat pokretanja (da li su svi testovi prošli) stoji u `README.md` datoteci repozitorijuma.

10.4 Primer funkcionisanja celokupnog sistema

TODO: napraviti dijagram sekvence koji se brine o tome sta zove sta, za ceo sistem

¹³<https://github.com/>

Glava 11

Zaključak

11.1 Zbog čega

TODO: zbog čega sam pravio ovaj sistem

11.2 Ograničenja sistema

TODO: ukratko sta je sve uradjeno, glavne limitacije sistema i sta raditi dalje (na gh-u je zapisano)

TODO: podrzano je i metadata Za indexe ali nije iskoristeno

Spisak slika

Slika 1	Uprošćena arhitektura sistema	1
Slika 2	Izgled <i>log</i> datoteke i smer iteracije	4
Slika 3	Izgled jednog bloka struktuiranog sa slogovima	9
Slika 4	Operacije, njihovi <i>log</i> tipovi i struktura <i>log</i> tipova	13
Slika 5	Hijerarhija konstanti	21
Slika 6	Hijerarhija izraza	22
Slika 7	Hijerarhija implementacije relacionih operatora	25
Slika 8	Primer poziva <code>getValue()</code> metode u konkretnom stablu relacionih operatora	28
Slika 9	Dijagram sekvence <code>next()</code> metode u konkretnom stablu relacionih operatora	29
Slika 10	Gramatika <code>Parse</code> sintaktičke kategorije	32
Slika 11	Gramatika <code>ParseUpdate</code> sintaktičke kategorije	33
Slika 12	Gramatika <code>ParseSelect</code> sintaktičke kategorije	33
Slika 13	Gramatika <code>ParseInsert</code> sintaktičke kategorije	34
Slika 14	Gramatika <code>ParseDelete</code> sintaktičke kategorije	34
Slika 15	Gramatika <code>ParseCreateTable</code> sintaktičke kategorije	35
Slika 16	Gramatika <code>ParsePredicate</code> sintaktičke kategorije	35
Slika 17	Gramatika <code>ParseExpression</code> sintaktičke kategorije	36
Slika 18	Hijerarhija implementacije planova	40
Slika 19	<i>Flowchart</i> računice redukcionog faktora člana	42
Slika 20	Struktura planera	47
Slika 21	<i>Flowchart</i> dijagram <code>BetterQueryPlanner</code> algoritma planiranja	49
Slika 22	Primer stabla planova nakon unije 3 naredbe	50
Slika 23	Primer stabla planova nakon filtriranja i projekcije	50
Slika 24	Primer stabla planova nakon proizvoda 3 tabele	51
Slika 25	Primer stabla planova nakon kvalifikacije kolona tabele	51
Slika 26	Primer kompletnog stabla planova	52
Slika 27	Struktura <code>Response</code> hijerarhije	58
Slika 28	Struktura <code>QuerySet</code> paketa	59
Slika 29	Dijagram sekvence obrade klijenta	60
Slika 30	Dijagram sekvence bezbednog gašenja servera	61

Spisak listinga

Listing 1	Identifikator bloka	3
Listing 2	Razičiti algoritmi izbora smene bafera	6
Listing 3	Opis jedne kolone	7
Listing 4	Podržani tipovi kolona	7
Listing 5	Fizičke karakteristike polja	8
Listing 6	Identifikator sloga	9
Listing 7	Prioriteti aritmetičkih operacija u sistemu	37
Listing 8	Primer rezultujuće tabele EXPLAIN naredbe	52
Listing 9	Primer ispisane tabele SELECT * FROM tablecatalog; naredbe	62
Listing 10	Konfiguracioni parametri JUnit biblioteke	63
Listing 11	Izgradnja svih artifakata sistema, bez pokretanja testova	65
Listing 12	Pokretanje svih testova u sistemu	65
Listing 13	Kod sistemske klase LBDBSettings	66

Spisak tabela

Tabela 1	Različiti izolacioni nivoi transakcija	16
Tabela 2	Šema <i>tablecatalog</i> tabele	19
Tabela 3	Šema <i>fieldcatalog</i> tabele	19
Tabela 4	Primer karakteristika tabela za računanje broja blokova plana proizvoda	44

Spisak korišćenih skraćenica

Skracenica**Opis**

WAL

Write Ahead Log (zapisivanje operacija unapred)

Spisak korišćenih pojmova

Pojam	Objašnjenje
SQL	Structured Query Language

Biografija

TODO: napisati biografiju

Literatura

- [1] E. Sciore, *Database Design and Implementation: Second Edition*. y Data-Centric Systems and Applications. Springer Cham, 2020.
- [2] T. Haerder и A. Reuter, „Principles of transaction-oriented database recovery“, *ACM computing surveys (CSUR)*, том 15, изд. 4, стр. 287–317, 1983.
- [3] P. A. Bernstein и N. Goodman, „Multiversion concurrency control—theory and algorithms“, *ACM Transactions on Database Systems (TODS)*, том 8, изд. 4, стр. 465–483, 1983.
- [4] P. Flajolet, É. Fusy, O. Gandouet, и F. Meunier, „Hyperloglog: the analysis of a near-optimal cardinality estimation algorithm“, *Discrete mathematics & theoretical computer science*, изд. Proceedings, 2007.
- [5] E. F. Codd, „A relational model of data for large shared data banks“, *Communications of the ACM*, том 13, изд. 6, стр. 377–387, 1970.
- [6] V. R. Pratt, „Top down operator precedence“, y *Proceedings of the 1st annual ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, 1973, стр. 41–51.
- [7] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, и T. G. Price, „Access path selection in a relational database management system“, стр. 23–34, 1979.
- [8] E. Wong и K. Youssefi, „Decomposition—a strategy for query processing“, *ACM Transactions on Database Systems (TODS)*, том 1, изд. 3, стр. 223–241, 1976.

Грешке у литературним наводима

Овде се налази списак грешака у литературним наводима које морате исправити у `literatura.bib` фајлу.

1. Референци `systemR` недостаје поље `journal`

TODOs

1. pretvoriti ovo u pravi flowchart
2. citirati detaljan dijagram u pregledu sistema
3. citirati performanse svih alg
4. citirati zaključak sa limitacijom sistema, fiksni nespanovani slogovi
5. citirati zaključak za limitaciju sistema racunanja statistickih podataka
6. limitaciju podrzavanja samo AND u zakljucku
7. Jedno od glavnih ograničenja sistema je da projektovane kolone ne mogu da sadrže logička računanja
8. citirati limitaciju unspanned sloga
9. `jline`
10. napraviti dijagram sekvence koji se brine o tome sta zove sta, za ceo sistem
11. zbog cega sam pravio ovaj sistem
12. ukratko sta je sve uradjeno, glavne limitacije sistema i sta raditi dalje (na gh-u je zapisano)
13. podrzano je i metadata Za indexe ali nije iskoristeno
14. napisati biografiju